

Domande orale di questo capitolo

mercoledì 31 maggio 2023 14:34

1. Che cosa è una macchina astratta? E in cosa si differenzia da una macchina fisica?

Dato un linguaggio di programmazione L , una macchina astratta M_L è un insieme di strutture dati e algoritmi per memorizzare ed eseguire programmi scritti in L .

Una macchina fisica è una particolare macchina astratta le cui strutture dati e algoritmi sono circuiti, porte logiche e dispositivi elettronici.

2. Che cos'è un interprete? In cosa consiste il ciclo fetch-decode-execute?

Un interprete scritto in un linguaggio L_0 e che interpreta un linguaggio L_1 è un programma che realizza una funzione parziale del tipo:

$$I_{L_1}^{L_0}(\text{Programma}^{L_1} \times D) \rightarrow D \quad \text{tale che} \quad I_{L_1}^{L_0}(P^{L_1}, D) = P^{L_1}(D)$$

L'interprete è il componente di una macchina astratta che serve per eseguire istruzioni. In particolare, contiene sistemi di gestione della memoria, controllo dei dati e controllo di flusso.

Il ciclo FDE è il ciclo eseguito da un interprete che consiste nel prelevare, decodificare ed eseguire un'istruzione.

3. Cos'è il linguaggio macchina?

Data una macchina astratta A , il linguaggio compreso dal suo interprete (L) è il suo linguaggio macchina.

4. Possono esistere macchine diverse con lo stesso linguaggio macchina?

Sì, in quanto dipende dal modo in cui l'interprete è implementato e le strutture dati che utilizza. Nel caso di una macchina hardware (come macchina astratta) si possono avere processori diversi che implementano lo stesso set di istruzioni.

5. In quali modi è possibile implementare una macchina astratta?

Hardware, software e firmware:

Realizzazione con **hardware**:

- È molto complicato implementare i costrutti di alto livello ma rende l'esecuzione di un programma scritto nel linguaggio della macchina astratta estremamente veloce, a scapito della flessibilità nel caso si dovesse modificare il linguaggio, perché significherebbe modificare l'intera architettura della macchina fisica.
- Lo si usa per linguaggi **di basso livello**.

Realizzazione con **software**:

- Ottima flessibilità ma bassa velocità. Comunque si basa su altre macchine astratte che dovranno essere implementate per poterlo effettivamente eseguire.

Realizzazione con **firmware**:

- Si tratta di una via di mezzo. Si ha una velocità maggiore del software ed una flessibilità maggiore dell'hardware, ma non una velocità maggiore dell'hardware e una flessibilità maggiore del software.
- Possibile grazie a microprogrammi scritti in linguaggi basso

6. Che cos'è un compilatore?

Un compilatore da L_0 a L_1 è un programma che realizza la funzione:

$$C_{L_0, L_1} : Prog_{L_0} \rightarrow Prog_{L_1}$$

Tale che, dato un programma P^{L_0} , se

$$C_{L_0, L_1}(P^{L_0}) = P^{L_1}$$

Allora, per ogni input, si ha:

$$P^{L_0}(Input) = P^{L_1}(Input)$$

7. Relativamente alla tecnica d'implementazione software, descrivere la tecnica d'implementazione interpretativa pura e quella compilativa pura.

Dato un linguaggio L_1 e una macchina astratta ospite per L_0 , un'implementazione può essere:

- **Compilativa pura** se si fornisce un compilatore C_{L_1, L_0} (Che viene eseguito su una qualsiasi macchina astratta in nostro possesso, non necessariamente M_0). Ogni istruzione di L_1 viene tradotta in una o più istruzioni di L_0 . In questo caso, quindi, la traduzione avviene interamente **prima** della fase di esecuzione.
- Interpretativa pura se si fornisce un interprete $I_{L_1}^{L_0}$, quindi scritto in linguaggio L_0 e che interpreta L_1 . Non vi è traduzione ma solo decodifica, che avviene durante l'esecuzione. Ad ogni istruzione L_1 viene assegnata una insieme di istruzioni L_0 che vengono eseguite **direttamente**.

8. Quando un interprete si può dire corretto? Quando un compilatore si può dire corretto?

Interprete o compilatori si possono dire corretti se preservano la semantica del linguaggio che hanno tradotto nel linguaggio nel quale traducono. Questo vuol dire che devono interpretare, o compilare, in un linguaggio "risultante" con uguale o maggiore espressività, in generale ci basta considerare la Turing Completezza (considerata la memoria infinita).

9. Confrontare l'implementazione di una macchina astratta su una macchina ospite per mezzo di un interprete o di un compilatore.

Se si sceglie l'**implementazione interpretativa** si ha una velocità minore, in quanto ai tempi di esecuzione del programma stesso si aggiungono i tempi di traduzione del codice sorgente. Non viene generato codice, in quanto il codice tradotto viene eseguito direttamente. Si può debuggare più facilmente in quanto si può capire qual è l'istruzione che è stata decodificata per ultima e cosa in questo ha dato un problema a livello di runtime. Si ha più flessibilità, per esempio per modificare a runtime il funzionamento del programma.

Se si sceglie invece **l'implementazione compilativa** si ha una maggiore velocità di esecuzione e ogni istruzione viene tradotta una sola volta, in fase appunto di compilazione. Si vanno a perdere dei vantaggi, come la possibilità di fare debugging in modo facile in quanto si vanno a perdere informazioni sul codice sorgente.

10. Come vengono implementate nella realtà le macchine astratte? Che cos'è la macchina intermedia?

Nella realtà, non si usa un approccio interpretativo o compilativo puro, bensì si va a fare un operazione di questo tipo:

Sia L il linguaggio del programma che dobbiamo eseguire e sia L_F il linguaggio della macchina fisica che abbiamo, noi: compiliamo L in L_i , dove L_i è un linguaggio intermedio, di una macchina astratta che si interpone tra quella fisica e quella astratta del linguaggio L , e poi usiamo un interprete da L_i a L_F .

11. Quando si dice che una implementazione è di tipo interpretativo e quando di tipo compilativo? Fare esempi di linguaggi la cui implementazione è di un tipo o dell'altro.

Si dice che sia di tipo interpretativo quando il linguaggio della macchina intermedia è molto lontano da quello della macchina fisica, ovvero c'è ancora lavoro di decoding da fare e da gestire e pertanto molto del carico ricade sull'interprete.

Un linguaggio che ha questo tipo di approccio è Java e la sua JVM, la quale è molto diversa dal linguaggio della macchina fisica sulla quale si appoggia e pertanto deve fare ancora un grande lavoro di traduzione in fase di esecuzione.

Si dice che sia invece di tipo compilativo quando il linguaggio della macchina intermedia è molto vicino, se non identico, a quello della macchina fisica. Di solito il linguaggio della macchina intermedia va ad aggiungere funzioni come la gestione dell'I/O che sono al di fuori di quello che la macchina fisica ha nel suo di linguaggio.

Un esempio è il C, il quale va ad eseguire una compilazione e durante l'esecuzione il binario generato è essenzialmente identico a quello della macchina fisica, se solo non fosse per queste gestioni che si devono aggiungere durante runtime.

12. L'interprete e il compilatore si possono sempre realizzare?

Sì, a patto che di avere a disposizione un linguaggio "espressivo" almeno tanto quanto quello da implementare. Questo si collega alla Turing-Completezza.

13. Che cos'è l'implementazione via kernel?

L'implementazione via kernel centra con un modo di generare un compilatore.

Se si deve implementare il linguaggio L in un linguaggio della macchina ospite L_0 :

- Si implementa un opportuno sottoinsieme H di L
- Si usa H come linguaggio di implementazione

Si implementa poi un compilatore oppure un interprete del tipo:

- C_{L,L_0}^H
- I_L^H

A questo punto, si deve implementare il linguaggio H piuttosto che l'intero linguaggio L, che, in quanto H è più piccolo, risulta molto più semplice.

Questo approccio è comune nei sistemi operativi, che creano inizialmente un nucleo e poi l'intero sistema usando le primitive offerte dal nucleo.

Ci semplifica l'implementazione in quanto H è più vicino di L al linguaggio della macchina ospite e ci facilita la portabilità, in quanto basta implementare H in un diverso linguaggio macchina.

14. Quando si parla di bootstrapping?

Si tratta di usare compilatori per creare nuovi compilatori, i quali compilano diversamente da quelli che li hanno generati.

Essenzialmente ci si riduce a creare (e necessitare) solo determinati strumenti (ovvero interpreti e compilatori) e il resto li andiamo a generare in modo "automatico" attraverso la compilazione dei compilatori.

Questo ci permette anche di creare nuovi compilatori in modo "automatico" in modo più facile, in quanto ci basta cambiare anche solo uno dei compilatori che noi abbiamo a disposizione (e facendolo in modo semplice, ovvero non andando magari a scrivere codice in binario ma comunque usando un linguaggio di alto livello).

"Il compilatore è scritto nel linguaggio che si intende compilare, poi viene tradotto usando interpreti o altri compilatori."

Domande orale di questo capitolo

04 June 2023 18:22

15. Quali sono i livelli di descrizione di un linguaggio?

Un linguaggio viene descritto su 4 livelli:

- Grammatica

Ovvero capire quali frasi sono corrette.

Si divide in due aspetti:

o Analisi lessicale

Usando un alfabeto sono riconosciute le corrette sequenze alfanumeriche che costituiscono i token (ovvero le parole) del linguaggio.

In un linguaggio comune, questo viene fatto con un dizionario.

Nei nostri casi, lo facciamo con le grammatiche regolari.

o Analisi sintattica

Ovvero capire quando una frase è corretta, ovvero **quali sequenze di token costituiscono frasi legali**.

La sintassi è quindi una relazione tra segni: tra tutte le sequenze di parole ne troviamo un sottoinsieme che costituiscono delle frasi del linguaggio.

- Semantica

Ovvero capire il significato delle frasi del linguaggio e quali di queste hanno "senso". Bisogna avere una base dalla quale si riesce a capire, non si può andare a tradurre il significato in modo ricorsivo per sempre: nel caso dei computer questa cosa è il linguaggio macchina, per le persone potrebbe essere per esempio la madrelingua.

È una relazione tra segni (ovvero le frasi corrette) e significati (entità autonome che esistono a prescindere da cosa usiamo per descriverle)

- Pragmatica

Altre regole che determinano come usare un linguaggio dal punto di vista pratico. Usare il lei al posto del tu in un contesto formale oppure evitare di usare il gergo, anche se il ciò è permesso da tutte le altre regole. Essenzialmente come usare una frase dentro un contesto.

- Implementazione

Ovvero, di un linguaggio di programmazione, come esso è effettivamente implementato, ovvero come viene eseguito da un punto di vista pratico, come le frasi "operative" del linguaggio realizzano lo stato di cui stanno parlando.

16. Sintassi: qual è l'aspetto lessicale e quale quello grammaticale di un linguaggio?

Vedere risposta scritta sopra.

17. Cos'è un alfabeto? Cos'è una parola o stringa? Cos'è A^* ? Tale insieme è enumerabile?

Un alfabeto è una collezione di simboli.

Una parola o stringa è un insieme di simboli che vengono concatenati.

A^* è la chiusura di Kleene dell'alfabeto A , che viene descritto nel seguente modo:

$$A^* = \bigcup_{n=0}^{\infty} A^n$$

Dove:

$$A^n = A \cdot A^{n-1} = \{aw \mid a \in A \wedge w \in A^{n-1}\}$$
$$A^0 = \{\varepsilon\}$$

18. Definizione di potenza di una stringa. Definizione di potenza di un linguaggio. Definizione di chiusura / iterazione (stella di Kleene) di un linguaggio

Potenza di una stringa:

$$w^n = \underbrace{w \cdot \dots \cdot w}_{n \text{ volte}}, \quad \text{per esempio } (ab)^3 = ababab$$

Potenza di un linguaggio:

$$L^n = L \cdot L^{n-1}, \quad L^0 = \{\varepsilon\}$$

$$\text{Dove la concatenazione funziona nel seguente modo: } L_1 \cdot L_2 = \{wv \mid w \in L_1 \wedge v \in L_2\}$$

Stella di Kleene

Scritta sopra

19. Definizione di grammatica libera da contesto. Come si deriva una stringa? Qual è il linguaggio generato da una grammatica libera?

Una grammatica libera da contesto è una quadrupla formata da:

(NT, T, R, S) , ovvero:

NT - Insieme dei simboli non terminali

T - Insieme dei simboli terminali

R - Insieme delle produzioni

Ovvero espressioni che sono nella forma $V \rightarrow w$, $V \in NT \wedge w \in \{T \cup NT\}^*$

$S \in NT$ - Simbolo iniziale

Fissata una grammatica $G = (NT, T, R, S)$ e due stringhe che bisogna verificare sia una la derivazione dell'altra $v \Rightarrow w$ ($v, w \in \{T \cup NT\}^*$), allora diciamo che $v \Rightarrow w$, ovvero v deriva immediatamente w , se:

$$\frac{v = xAy \quad (A \Rightarrow z) \in R \quad w = xzy}{v \Rightarrow w}$$

Vi è poi la definizione di "deriva", che è la chiusura transitiva e riflessiva della derivazione immediata, ovvero:

$$\frac{}{v \Rightarrow^* v}, \quad \frac{v \Rightarrow^* w' \quad w' \Rightarrow w}{v \Rightarrow^* w}$$

Il linguaggio generato da una grammatica libera $G = (NT, T, R, S)$ è l'insieme:

$L[G] = \{w \in T^* \mid S \Rightarrow^* w\}$, ovvero tutte le stringhe formate unicamente da simboli terminali che si possono produrre partendo dal simbolo iniziale.

20. Cos'è un albero di derivazione? Cos'è una derivazione canonica sinistra / destra? Esiste una corrispondenza biunivoca tra alberi di derivazioni e derivazioni canoniche?

Data una grammatica libera $G = (NT, T, R, S)$, un **albero di derivazione** è un albero ordinato in cui:

1. Ogni nodo è etichettato con un simbolo $T \cup NT \cup \varepsilon$
2. La radice è etichettata con S
3. Ogni nodo interno è etichettato con un simbolo in NT
4. Se il nodo n
 - a. Ha etichetta $A \in NT$ e l'insieme delle etichette dei suoi figli è $\{X_i \in NT \cup T \mid \forall i \in [1, k]\}$, allora esiste una produzione $A \rightarrow X_1 \dots X_k \in R$
 - b. Se il nodo ha etichetta ε , allora n è una foglia, è l'unico figlio, e sia detto A suo padre, allora $A \rightarrow \varepsilon$ è una produzione in R
5. Se inoltre ogni nodo foglia è etichettato su $T \cup \varepsilon$, allora l'albero di derivazione si dice sia completo.

Una derivazione sinistra vuol dire che viene espanso sempre il non terminale più a sinistra usando una produzione. Quella destra è uguale ma espande quello più a destra. Derivazioni destre e sinistre possono generare gli stessi alberi.

Questa corrispondenza esiste:

- Ogni derivazione è associata ad un albero di derivazione ed ogni albero di derivazione è associato ad una derivazione.

21. Quando una grammatica è ambigua? (fare un esempio) Quando linguaggio è ambiguo? (fare un esempio).

Una grammatica è ambigua quando vi sono due produzioni che danno la stessa stringa ma due alberi di derivazione distinti.

Un esempio di grammatica ambigua è:

$$(A, a, \{A \rightarrow A + A \mid A - A\}a, A)$$

Un linguaggio è ambiguo quando ogni sua grammatica risulta ambigua. Questi linguaggi si dicono inerentemente ambigui.

Non vi sono linguaggi regolari che sono inerentemente ambigui.

Un esempio di linguaggio ambiguo è:

$$\{a^n b^m c^m \mid m, n \geq 1\} \cup \{a^m b^m c^n \mid m, n \geq 1\}$$

22. È possibile rimuovere l'ambiguità dalla grammatica delle espressioni aritmetiche? Come?

Vengono fatte due cose:

- Dare una certa precedenza agli operandi (Per esempio indicare che il * ha una precedenza maggiore del +). Questo genera dell'associatività intrinseca a come una stringa esce
- La prima operazione non basta per mantenere tutta l'espressività delle espressioni aritmetiche: si aggiungono le parentesi, che fungono da zucchero sintattico, ovvero servono per dare tutto il potere espressivo al linguaggio. Poi queste verranno tolte dall'effettivo albero sintattico che verrà generato.

23. Cos'è l'albero di sintassi astratta? Che differenza c'è tra sintassi concreta e sintassi astratta? Cos'è lo zucchero sintattico?

L'albero di sintassi astratta è un albero di derivazione che soddisfa i vincoli contestuali di un linguaggio in cui compaiono solo i terminali.

La sintassi concreta è una grammatica non ambigua che fa uso di zucchero sintattico. La sintassi astratta è una grammatica semplice ed intuitiva, ma ambigua. Dall'albero di derivazione da sintassi concreta ne estraiamo un albero sintattico da sintassi astratta, detto albero sintattico astratto.

Per zucchero sintattico si intendono simboli atti solo a chiarire la derivazione di una stringa ed è essenzialmente ciò che differenzia la sintassi astratta e la sintassi concreta di un linguaggio.

24. Fare esempi di vincoli sintattici contestuali. Possono essere catturati attraverso grammatiche libere?

Un esempio di vincolo sintattico contestuale è per esempio capire se il numero di parametri formali coincide con il numero di parametri attuali di una funzione e della sua chiamata. Dobbiamo sapere, appunto, quanti questi sono ed il ciò dipende dal contesto nel quale siamo.

Un'altro esempio è che un identificatore deve essere dichiarato prima dell'uso.

Un'altro ancora è che prima di usare una variabile ci deve essere un assegnamento su di essa.

Questo non può essere catturato attraverso le grammatiche libere, ma solo attraverso:

- Grammatiche dipendenti da contesto, molto complesse da scrivere
- Metodi ad hoc, che sistemano il dubbio attraverso controlli specifici.

25. Cosa s'intende per semantica statica? E per semantica dinamica?

Praticamente, nei linguaggi di programmazione, si va a fare una distinzione all'interno della grammatica: anche controlli che dovrebbero essere eseguiti nella grammatica, ovvero la correttezza delle frasi, vengono "posticipati" per essere fatti dall'analizzatore semantico.

All'interno dei linguaggi di programmazione pertanto:

- Analisi sintattica:
 - o Tutto ciò che fa una BNF
- Analisi semantica:
 - o Analisi sintattica contestuale
 - o E anche altro

Quindi, all'interno di questa seconda fase, si differenziano:

- Semantica statica
 - o Ovvero tutti quei controlli che possono essere eseguiti prima dell'esecuzione,

- analizzando il codice del programma
- Semantica dinamica
 - Ovvero vengono eseguiti controlli su una rappresentazione formale dell'esecuzione del programma, che possono mostrare problemi durante l'esecuzione effettiva di esso.
 - Per esempio, dividere per l'input dell'utente, quando questo input potrebbe essere 0, è un errore di semantica dinamica.

26. Elencare le varie fasi in cui si articola un compilatore. Descrivere in dettaglio ogni singola fase.

Vi sono 6 fasi:

- Analisi lessicale
 - In questa fase si controlla che tutte le singole parole siano corrette e si vanno a generare dei token che vengono aggiunti alla tabella dei simboli
- Analisi sintattica
 - Vengono controllate le "frasi" e si generano gli alberi di derivazione
 - Ogni foglia di tale albero è costituita da un token contenuto nella lista e leggendo da sinistra a destra le foglie dell'albero si deve ottenere una frase legale per il linguaggio
 - Si eseguono controlli sul numero di parentesi, sulla struttura del codice (if,else,loop etc.) e se rispettano come dovrebbe venire scritto
- Analisi semantica
 - Vengono eseguiti controlli di semantica statica, ovvero si verificano se i parametri attuali corrispondono con quelli formali, che le variabili hanno un tipo, etc.
 - Viene arricchito l'albero di derivazione con informazioni sui tipi dei token e altre informazioni.
- Codice intermedio
 - Viene generato un codice intermedio che è sia indipendente dall'architettura che dal sorgente.
 - È formato genericamente da operazioni con 3 operandi ($x = y + z$ per esempio)
 - È facile da produrre, in quanto segue la struttura dell'albero sintattico, ed è facile da tradurre, in quanto è molto più semplice
- Ottimizzazione forma intermedia
 - Rimozione di codice inutile
 - Espansioni in-line di chiamate di funzione
 - Fattorizzazione di sottoespressioni per non calcolare più volte lo stesso valore
 - Mette fuori dai cicli le sottoespressioni che non variano
 - Altro, dipendentemente dal compilatore
- Generazione del codice
 - Viene generato il codice oggetto per una specifica architettura (include assegnazione dei registri e ottimizzazioni specifiche per l'architettura e il codice oggetto.)

27. A chi serve definire la semantica di un linguaggio e perché?

Definire la semantica di un linguaggio di programmazione è molto più difficile della sintassi. La complessità nasce dall'esigenza di mediare tra due istanze contrapposte: la ricerca dell'esattezza, da una parte, e quella della flessibilità dall'altra in modo da rimuovere l'ambiguità per un utente, ma anche da lasciare spazio all'implementazione.

A chi serve la semantica dinamica?

- Al programmatore
 - Analisi del programma
 - Deve sapere esattamente cosa debba fare il suo programma

- Deve poter dimostrare proprietà del suo programma ("termina sempre per ogni input",...)
- Al progettista del linguaggio
 - Strumento di specifica del linguaggio
 - Deve poter dimostrare proprietà del linguaggio ("è Turing completo?",...)
- All'implementatore del linguaggio
 - Riferimento per dimostrare la correttezza dell'implementazione

28. Quali tecniche si usano per dare semantica ad un linguaggio di programmazione?

Vi sono 2 tecniche:

- **Operazionale:** (Macchina astratta a stati e transizioni)
 - Si costruisce una specie di automa che, passo a passo, mostra l'effetto della esecuzione delle varie istruzioni
 - Enfasi su **come** si calcola
 - Vedremo l'uso di questa semantica per un semplice linguaggio imperativo
- **Denotazionale:**
 - Si associa ad ogni programma sequenziale una funzione da input e un output (incluse strutture ausiliarie dette ambiente e memoria)
 - Enfasi su **cosa** si calcola (vengono nascosti i passi intermedi del calcolo)

29. Imparare le regole di semantica operativa SOS per il semplice linguaggio presentato a lezione. Regole di valutazione interna-sinistra ed esterna-sinistra.

Tante cose da scrivere, non ho voglia

30. Cosa s'intende per pragmatica? Cosa si intende per implementazione di un linguaggio?

Pragmatica:

Insieme di "regole" e consigli su come usare un linguaggio:

- Usare "while" per cicli indeterminati, "for" per cicli predeterminati.
- Non usare il "go-to" per saltare da una parte all'altra
- ...

Implementazione:

Ovvero la scrittura di un compilatore (o un interprete) per una macchina ospite già realizzata, costruendo così una macchina astratta per il linguaggio.

- Correttezza dell'implementazione?
 - Bisogna dimostrare che preserva la semantica del linguaggio, ovvero che il programma sorgente e quello oggetto calcolano la stessa funzione (per i linguaggi sequenziali).
- Costo dell'implementazione
 - È il compilatore in grado di produrre codice efficiente?
 - Certi costrutti linguistici, scelti dal progettista del linguaggio, sono troppo onerosi in pratica?

Domande orale di questo capitolo

lunedì 5 giugno 2023 11:34

31. Cosa fa l'analizzatore lessicale?

L'analizzatore lessicale, anche chiamato scanner, prende in input il codice e ne restituisce l'elenco dei token. In particolare, associa ogni token ad una "categoria", e questo viene dedotto usando il pattern associato ad essa.

Altra risposta: scopo dell'analizzatore lessicale è quello di riconoscere nella stringa in ingresso alcuni gruppi di caratteri che corrispondono a certe categorie sintattiche. In tal modo la stringa in ingresso è trasformata in una sequenza di simboli astratti, detti token, che sono poi passati all'analizzatore sintattico.

32. Cos'è un token?

È un'informazione astratta che rappresenta una stringa del testo in ingresso. È una coppia (nome, valore): il nome del token è un simbolo astratto che rappresenta una categoria sintattica, il valore invece è costituito da una sequenza di simboli del testo in ingresso.

33. Cos'è un pattern? Come lo si rappresenta? Cos'è un lessema?

Un pattern è una descrizione del linguaggio al quale un token appartiene. Questa descrizione viene fatta usando le espressioni regolari. Un lessema è una particolare istanza di un pattern, ovvero una stringa appartenente al linguaggio.

34. Definizione di espressioni regolari (sintassi) e di linguaggio associato (semantica)

Le espressioni regolari si possono definire attraverso la seguente BNF:

$$r := \emptyset \mid \varepsilon \mid r \cdot r \mid r|r \mid r^* \mid \bigcup_{a \in A} a$$

Abbiamo inoltre che:

- Sono tutte associative a sinistra
- La precedenza è $*$ $>$ $\cdot$ $>$ $|$

Il linguaggio associato invece ad un'espressione regolare è una funzione del tipo:

$$\mathcal{L} : Exp.reg \mapsto \mathcal{P}(A^*)$$

E funziona nel seguente modo:

$$\begin{aligned}\mathcal{L}[\emptyset] &= \emptyset \\ \mathcal{L}[\varepsilon] &= \{\varepsilon\} \\ \mathcal{L}[a] &= \{a\}, \quad \forall a \in A \\ \mathcal{L}[r \cdot r] &= \mathcal{L}[r] \cdot \mathcal{L}[r] \\ \mathcal{L}[r|r] &= \mathcal{L}[r] \cup \mathcal{L}[r] \\ \mathcal{L}[r^*] &= \bigcup_{n=0}^{\infty} \mathcal{L}[r]^n\end{aligned}$$

$$\mathcal{L}[(r)] = \mathcal{L}[r]$$

35. Quali sono i linguaggi regolari? I linguaggi finiti sono tutti regolari? Esistono linguaggi infiniti regolari?

I linguaggi regolari sono tutti quei linguaggi per cui esiste un'espressione regolare associata a tale linguaggio.

I linguaggi finiti sono regolari in quanto ci basterebbe mettere un "or" tra tutte le singole stringhe.

Esistono linguaggi infiniti regolari, questi vengono generati quando si usa la $*$.

36. Definizione di equivalenza tra espressioni regolari. Elencare alcune leggi di equivalenza.

Due espressioni regolari si dicono equivalenti se vengono associate allo stesso linguaggio regolare.

Alcune leggi di equivalenza potrebbero essere:

$$\begin{aligned} r|r &\cong r \\ a^*|\varepsilon &\cong a^* \\ r^{**} &\cong r^* \\ r \cdot \varepsilon &\cong \varepsilon \cdot r \cong r \end{aligned}$$

37. Cos'è una definizione regolare e a cosa serve?

È una lista di definizioni di simboli in cui, ad ogni simbolo, è associato un'espressione regolare. Praticamente noi andiamo a definire:

$$d_1, \dots, d_k \quad \text{dove} \quad d_1 = r_1, \quad \dots, \quad d_k = r_k$$

Dove $r_1, \dots, r_k$ sono espressioni regolari e $d_1, \dots, d_k$ sono definizioni.

All'interno delle espressioni regolari che usano queste definizioni (tra cui anche $r_1, \dots, r_k$) l'alfabeto diventa $A \cup \{d_1, \dots, d_k\}$ (Ovvero possiamo usare i nuovi nomi dati alle definizioni all'interno delle altre espressioni regolari come simboli).

È utile per esprimere espressioni regolari complesse, in quanto le si divide in sotto-espressioni in cui ognuna è associata ad un simbolo.

38. Definizione di NFA (automa finito non deterministico). Discutere cosa si intenda per non deterministico. Mettere in relazione la definizione formale con la rappresentazione grafica come diagramma di transizioni.

Un NFA è una quintupla:

$$(\Sigma, Q, \delta, q_0, F)$$

Dove:

- Σ è l'insieme di simboli dell'alfabeto

- Q è l'insieme di stati
- δ è una funzione di transizione, del tipo:
 - $\delta : Q \times (\Sigma \times \{\varepsilon\}) \mapsto \mathcal{P}(Q)$
- $q_0 \in Q$ è lo stato iniziale
- $F \subseteq Q$ è l'insieme di stati finali.

Per non deterministico si intende che con gli stessi input, l'automa può dare risultati anche diversi. Questo succedere in due casi: o ci sono più nodi per lo stesso simbolo, oppure si usa il simbolo ε per la transizione, che non consuma input. Questo vuole anche dire che non si può con certezza in modo immediato se una parola appartiene o meno in quanto si potrebbero incorrere in degli errori a volte e potrebbe andare bene in altre, quindi avviene spesso il backtracking per andare a fare tutti i tentativi possibili.

L'ultima parte della domanda è facilissima e lunghissima quindi skip.

39. Come si definisce il linguaggio accettato da un NFA? Definizione di equivalenza tra NFA.

Il linguaggio accettato dall'NFA N è:

$$L[N] = \{w \in \Sigma^* \mid (q_0, w) \vdash_N^* (q, \varepsilon) \wedge q \in F\}$$

Dove $(q_0, w) \vdash_N^* (q, \varepsilon)$ vuol dire che c'è un insieme di mosse che si possono arrivare per passare da uno stato dove si deve consumare la stringa w ad uno stato dove si è consumata tutta la stringa.

Questo viene definito come:

$$\frac{}{(q_0, w) \vdash_N^* (q_0, w)}, \quad \frac{(q, w) \vdash_N^* (q', w') \quad (q', w') \vdash_N (q'', w'')}{(q, w) \vdash_N^* (q'', w'')}$$

Dove la $\vdash_N$ senza asterisco è definita come la mossa ed è:

$$\frac{q' \in \delta(q, a)}{(q, aw) \vdash_N (q', w)}$$

Due NFA sono equivalenti se entrambi accettano lo stesso linguaggio, ovvero:

$$\text{Siano } N, N' \text{ due NFA, allora } N \text{ equivalente } N' \Leftrightarrow L[N] = L[N']$$

40. Definizione di DFA (automa finito deterministico). Discutere cosa si intende per deterministico. Mostrare che un DFA è un caso speciale di NFA, ovvero la classe dei DFA è un sottoinsieme della classe degli NFA.

Un DFA è una quintupla definita come segue:

$$(\Sigma, Q, \Delta, q_0, \mathcal{F})$$

Dove:

- Σ è l'insieme dei simboli dell'alfabeto
- Q è l'insieme degli stati
- Δ è una funzione del tipo:

- $\Delta : T \times \Sigma \mapsto T$
- q_0 è lo stato iniziale
- $\mathcal{F}$ è l'insieme degli stati finali

Quindi è identico ad un automa non deterministico se non per la sua funzione di transizione Δ .

È deterministico in quanto per ogni singolo simbolo in Σ esiste una sola mossa possibile. Questo vuol dire che, dato lo stesso input, si avrà sempre lo stesso risultato, ovvero riconosciuto oppure non riconosciuto, per ogni automa DFA e per ogni input.

Un DFA è un caso speciale degli NFA in quanto non vi sono archi etichettati con ε e inoltre nelle funzioni di transizione si ha che $\Delta(q, a)$ è sempre esattamente costituita da un solo stato. Gli NFA hanno quindi più libertà nella loro scrittura dei DFA, quindi i DFA sono un caso speciale.

41. Dato un NFA, come si ricava un equivalente DFA? Ovvero descrivere come è definita la costruzione per sottoinsiemi. Definizione di epsilon-closure e algoritmo associato. Qual è la complessità della costruzione per sottoinsiemi nel caso pessimo? Ovvero se NFA ha n stati, quanti stati può avere il DFA equivalente?

A partire da un NFA, per trovare un DFA si usa la **costruzione per sottoinsiemi**, la quale internamente fa uso della **epsilon-closure**. Quindi definiamo questi due algoritmi:

- Epsilon-closure di q :

```

Impostare  $\varepsilon - \text{closure}(q) = \{q\}$ 
Impostare  $T = \{q\}$ 
while  $T \neq \emptyset$  do:
    get  $x = \text{pop}(T)$ 
     $\forall z \in \delta(x, \varepsilon) \notin \varepsilon - \text{closure}(q)$ 
        add  $z$  in  $\varepsilon - \text{closure}(q)$ 
    add  $z$  in  $T$ 

```

- Costruzione per sottoinsiemi: sia N un NFA definito da $(\Sigma, Q, \delta, q_0, F)$

```

Impostare  $A = \varepsilon - \text{closure}(q_0)$ 
Impostare  $T = \{A\}$ 
while  $X \in T$  non marcato
    marca  $X$ 
     $\forall \sigma \in \Sigma$  do
        if  $\delta(X, \sigma) \notin T$ 
            sia  $Y = \varepsilon - \text{closure}(\text{mossa}(X, \sigma))$ 
            add  $Y$  in  $T$ 
        Definisci  $\Delta(X, \sigma) = Y$ 

```

Definiamo matematicamente l'epsilon closure:

$$\frac{}{q \in \varepsilon - \text{closure}(q)}, \quad \frac{z \in \varepsilon - \text{closure}(q)}{\delta(z, \varepsilon) \subseteq \varepsilon - \text{closure}(q)}$$

La mossa invece è:

$$mossa : \mathcal{P}(Q) \times \Sigma \mapsto \mathcal{P}(Q)$$

$$mossa(P, a) = \bigcup_{\forall p \in P} \delta(p, a)$$

Nel caso pessimo, se un NFA ha n stati, allora il DFA creato usando la costruzione per sottoinsiemi ha 2^n stati, quindi è esponenziale.

- 42. Dato i due punti precedenti, enunciare il teorema che dice che la classe dei linguaggi riconosciuti da NFA coincide con la classe dei linguaggi riconosciuti da DFA.**

Sia N un NFA e sia M un DFA costruito partendo da N usando la costruzione per sottoinsiemi. Allora:

$$L[N] = L[M]$$

Per il corollario si ha che la classe dei linguaggi riconosciuti dagli NFA è uguale a quella riconosciuta dai DFA.

- 43. Come si costruisce un NFA a partire da una espressione regolare, in modo che il linguaggio riconosciuto dall'NFA sia lo stesso del linguaggio associato all'espressione regolare?**

Si costruisce nel seguente modo:

(Si vanno a definire dei modi per costruire un NFA per ogni singola operazione della grammatica regolare. In seguito, si va a dimostrare che il linguaggio è equivalente per questi, allora induttivamente è equivalente per qualsiasi trasformazione che li usa).

Questi mattoncini sono:

$$\bullet S = \emptyset \quad N[S] = \rightarrow 0 \quad \textcircled{0}$$

$$\text{Osserva che } L[\emptyset] = \emptyset = L[N[S]]$$

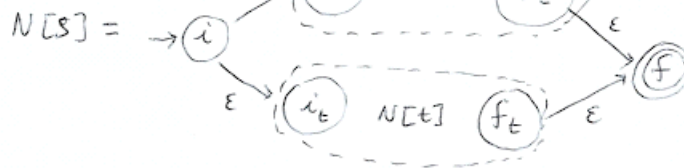
$$\bullet S = \epsilon \quad N[S] = \rightarrow 0 \xrightarrow{\epsilon} \textcircled{0}$$

$$\text{Osserva che } L[\epsilon] = \{\epsilon\} = L[N[S]]$$

• $S = a$ $N[S] = \rightarrow 0 \xrightarrow{a} 0$ (32)

Osserva che $L[a] = \{a\} = L[N[S]]$

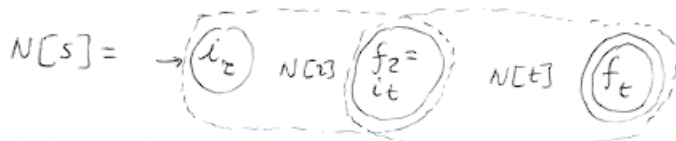
• $S = r | t$



Osserva che

$L[r | t] = L[r] \cup L[t] \stackrel{\text{per ipotesi induttiva}}{=} L[N[r]] \cup L[N[t]] = L[N[r | t]]$

• $S = r \cdot t$



Abbiamo fuso insieme il finale di $N[r]$ con l'iniziale di $N[t]$

Osserva che

$L[r \cdot t] = L[r] \cdot L[t] \stackrel{\text{per ipotesi induttiva}}{=} L[N[r]] \cdot L[N[t]] = L[N[r \cdot t]]$

• $S = r^*$



Osserva che

$L[r^*] = (L[r])^* \stackrel{\text{per ipotesi induttiva}}{=} (L[N[r]])^* = L[N[r^*]]$

per ipotesi induttiva

c.v.d

Una volta che si hanno questi, di un'espressione regolare si crea l'albero di derivazione e si parte dalle foglie, come nell'esempio successivo:

$$S = (a|b)^* ba$$

Partiamo dalle
foglie e risaliamo
alla radice
(bottom-up)



albero
sintattico

$$N[a] \quad \circ \xrightarrow{a} \odot$$

$$N[b] \quad \circ \xrightarrow{b} \odot$$

$$N[a|b] \quad \begin{array}{c} \circ \xrightarrow{\epsilon} \circ \xrightarrow{a} \circ \xrightarrow{\epsilon} \odot \\ \circ \xrightarrow{\epsilon} \circ \xrightarrow{b} \circ \xrightarrow{\epsilon} \odot \end{array}$$

$$N[(a|b)^*] \quad \begin{array}{c} \circ \xrightarrow{\epsilon} \circ \xrightarrow{\epsilon} \circ \xrightarrow{a} \circ \xrightarrow{\epsilon} \odot \\ \circ \xrightarrow{\epsilon} \circ \xrightarrow{\epsilon} \circ \xrightarrow{b} \circ \xrightarrow{\epsilon} \odot \end{array}$$

$$N[ba] \quad \circ \xrightarrow{b} \circ \xrightarrow{a} \odot$$

$$N[S] \quad \begin{array}{c} \rightarrow \circ \xrightarrow{\epsilon} \circ \xrightarrow{\epsilon} \circ \xrightarrow{a} \circ \xrightarrow{\epsilon} \odot \\ \rightarrow \circ \xrightarrow{\epsilon} \circ \xrightarrow{\epsilon} \circ \xrightarrow{b} \circ \xrightarrow{\epsilon} \odot \end{array}$$

44. Definizione di grammatica regolare:

Una grammatica regolare è una grammatica libera da contesto, quindi una quadrupla del tipo (NT, T, R, S) dove ogni produzione in R è del tipo:

$$A \rightarrow aX, \quad \forall a \in T, \forall A, X \in NT$$

oppure

$$A \rightarrow a, \quad \forall a \in T, \forall A \in NT$$

Inoltre, può esistere la produzione ϵ solo da S , quindi:

$$S \rightarrow \epsilon \quad \text{è ammessa.}$$

Per la definizione più lasca, la ϵ - production può essere presente in qualsiasi non terminale.

45. Come si associa ad una grammatica regolare un equivalente NFA?

Si deve prima costruire l'espressione regolare e poi si va a costruire un NFA usando ciò che vi è scritto sopra.

Si possono anche associare ai non terminali gli stati e far sì che le transizioni siano la lettera nella produzione.

Per le produzioni del tipo $A \rightarrow a$, la transizione $\delta(A, a) = X \in F$
 Se vi è la produzione ε allora S è uno stato finale.

46. Dato un DFA, come si costruire una grammatica regolare equivalente?

Dato un DFA $M=(\Sigma, Q, \delta, q_0, F)$, si costruisce la grammatica $G_M = (Q, \Sigma, R, q_0)$ nel seguente modo:

- Per nonterminali gli stati di M
- Per terminali, l'alfabeto di M
- Per simbolo iniziale, lo stato iniziale di M
- Per le produzioni R :
 - $\forall \delta(q_i, a) = q_j$, la produzione $q_i \rightarrow aq_j \in R$
 - se $\delta(q_i, a) = q_j \in F$, allora $q_i \rightarrow a \in R$
 - se $q_0 \in F$, allora $q_0 \rightarrow \varepsilon \in R$

Si può anche costruire in modo più lasco dicendo anche che:

- $\forall \delta(q_i, a) = q_j$ la produzione $q_i \rightarrow aq_j \in R$
- se $q \in F$, allora $q \rightarrow \varepsilon \in R$

47. Data una grammatica regolare, come si costruire un'espressione regolare equivalente?

In generale, sia data la grammatica regolare del tipo:

$$\begin{aligned} A_1 &= a_{1,1}A_1 \mid \dots \mid a_{1,n}A_n \mid b_{1,1} \mid \dots \mid b_{1,p_1} \\ A_2 &= a_{2,1}A_1 \mid \dots \mid a_{2,n}A_n \mid b_{2,1} \mid \dots \mid b_{2,p_2} \\ &\dots \\ A_n &= a_{n,1}A_1 \mid \dots \mid a_{n,n}A_n \mid b_{n,1} \mid \dots \mid b_{n,p_n} \end{aligned}$$

Si parte con creare:

$$A_n = S_n[A_1, \dots, A_{n-1}]$$

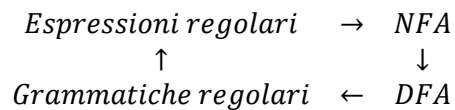
Ovvero si genera l'espressione regolare contenente i nonterminali fino ad A_{n-1} e la si sostituisce alla definizione di A_n .

Poi si fa la stessa cosa per A_{n-1}

$$A_{n-1} = S_n[A_1, \dots, A_{n-2}]$$

E così via, fino ad arrivare ad A_1 . A questo punto, usando equivalenze, ci si può ridurre ad una espressione regolare.

48. Descrivere, con un diagramma riassuntivo, tutte le relazioni fra i formalismi introdotti: NFA, DFA, grammatiche regolari, espressioni regolari. Questo diagramma dimostra che tutti questi formalismi sono equivalenti e descrivono la classe dei linguaggi regolari.



Conseguenza:

Tutti questi formalismi sono equivalenti.

Tutti generano / riconoscono / descrivono la stessa classe di linguaggi, ovvero i **linguaggi regolari**

$\text{gram.reg} \quad \text{NFA/DFA} \quad \text{exp.reg}$

Queste cose vengono dimostrate perché:

Da 1 a 2 abbiamo una costruzione (mattoncini che poi si uniscono a partire dalle foglie dell'albero di derivazione)

Da 2 a 3 abbiamo una costruzione (costruzione per sottoinsiemi)

Da 3 a 4 abbiamo una costruzione (una per definizione rigorosa e una per definizione lasca)

Da 4 a 1 abbiamo una costruzione (sostituzione e incrociare le dita)

49. Definizione di stati equivalenti in un DFA. Quando due stati di un DFA sono distinguibili?

Due stati $q_1, q_2 \in Q$ di un DFA sono equivalenti se e solo, $\forall x \in \Sigma^*, \hat{\delta}(q_1, x) \in F \Leftrightarrow \hat{\delta}(q_2, x) \in F$. Si può anche scrivere come: $L[N, q_1] = L[N, q_2]$, dove N è il DFA al quale q_1 e q_2 appartengono.

Due stati $q_1, q_2 \in Q$ sono distinguibili se $\exists x \in \Sigma^*$ tale che:

$$\hat{\delta}(q_1, x) \in F \wedge \hat{\delta}(q_2, x) \notin F$$

Oppure

$$\hat{\delta}(q_1, x) \notin F \wedge \hat{\delta}(q_2, x) \in F$$

50. Come funziona l'algoritmo iterativo con tabella a scala per produrre le classi di equivalenza di stati di un DFA?

Definiamo la relazione di equivalenza fino ad un stringa di lunghezza i come:

$$\sim_0 \subseteq F \times F \cup \{Q \setminus F \times Q \setminus F\}$$

$$q_1 \sim_i q_2 \Leftrightarrow \delta(q_1, a) \sim_{i-1} \delta(q_2, a), \quad \forall a \in \Sigma$$

L'algoritmo iterativo con tabella a scala funziona nel seguente modo:

Costruire una tabella a scala

Marcare con x_0 tutte le coppie che appartengono a $\sim_0$

$b = \text{true}$

for $i > 0$ e $b \neq \text{false}$:

$b = \text{false}$

$\forall a \in \Sigma$ do

$\forall (q, q')$ non marcate, if $(\delta(q, a), \delta(q', a))$ sono segnati con x_i

$\text{segna } (\delta(q, a), \delta(q', a)) \text{ con } x_{i+1}$
 $b = \text{true}$
 $i++$

51. Una volta determinate le classi di equivalenza degli stati di un DFA, come si costruisce l'automa minimo associato?

Dopo che abbiamo determinato le classi minime di un DFA $M = (\Sigma, Q, \delta, q_0, F)$, il DFA minimo associato, sia esso $M_m = (\Sigma, Q_m, \delta_m, [q_0], F_m)$ dove:

$$\begin{aligned}
 Q_m &= \{[q] \mid q \in Q\} \text{ con } [q] = \{q' \in Q \mid q \sim q'\} \\
 \delta_m([q], a) &= [\delta(q, a)] \\
 F_m &= \{[q] \mid q \in F\}
 \end{aligned}$$

52. Cos'è Lex? Qual è il suo input e il suo output?

Lex è un generatore di scanner, ovvero di analizzatori lessicali.
 Richiede in input un file ".l" che è formato da un insieme di definizioni regolari e da un insieme di azioni da eseguire.
 L'output è un file ".c" che si può compilare e, una volta compilato, si può usare per scannare un file.
 Esso è composto da automi riconoscitori delle definizioni regolari inserite e che esegue le azioni elencate

53. Qual è la struttura di un file ".l"? Come funziona l'analizzatore lessicale prodotto da Lex?

È formato da 3 parti:

(1) Dichiarazioni (ovvero definizioni regolari) della forma:

```

cirfra [0 - 9]
cifre cifra +
ide [a - zA - Z]([a - zA - Z][0 - 9])*
  
```

%% (separatore tra dichiarazioni e azioni)

(2) Azioni, del tipo: {exp.regolare} {azione}

```

{cifre} {printf("<Num,%s", yytext)}
  
```

Dove yytext viene sostituito con il valore

%%

(3) Funzioni ausiliare

Nel caso quelle inserite nelle azioni fossero molto complicate e si volessero definire in modo più chiaro.

Il programma C ottenuto, e compilato per renderlo eseguibile, implementa essenzialmente il DFA riconoscitore dell'insieme delle espressioni regolari contenute nell regole:

- Scandisce il testo sorgente alla ricerca di una stringa che corrisponda a (cioè sia un lessema

- per) una delle espressioni regolari (pattern di categoria sintattica)
- Quando riconosce un lessema, esegue l'azione specificata, e passa in output il risultato dell'azione al posto del lessema
- Quando l'input non corrisponde a nessun pattern, lo lascia inalterato e "segnala" la cosa al "gestore degli errori".

54. Come si interfaccia Lex con Yacc?

Il programma generato da LEX non è usato da solo, ma viene chiamato come subroutine da YACC (un generatore di analizzatori sintattici).

Alcune variabili comuni permettono di scambiare dati.

lex.yy.c viene usato "on-demand" da YACC per richiedere il token successivo.

yyllex() restituisce il nome del token, mentre il valore del token è condiviso nella variabile *yylval*.

55. Intestazione e dimostrazione del pumping lemma.

$L \text{ regolare} \Rightarrow \exists N \in \mathbb{N} : \forall w \in L : |w| \geq N \Rightarrow$

$\exists u, v, z \text{ tali che } w = uvz$

$|v| \geq 1$

$|uv| \leq N$

$\forall k \in \mathbb{N}, \quad uv^kz \in L$

Dimostrazione

Se si considera una grammatica regolare L , allora si può considerare un DFA N minimizzato associato ad essa. Siano il numero di stati di questo DFA $= N$.

Allora prendiamo una stringa appartenente a L lunga N , essa percorre $N + 1$ stati.

Sicuramente, pertanto, ha ripercorso uno degli stati in un ciclo. Questo ciclo, pertanto, può essere sia saltato ($k=0$), sia ripetuto n volte ($k \in \mathbb{N}$) e la parola verrebbe ugualmente riconosciuta.

Notiamo che questa parte ripetuta è almeno lunga 1, in quanto si trova effettivamente nella stringa.

Inoltre notiamo che si troverà sicuramente prima di N o ad N , non si potrà trovare dopo N , in quanto l'automa ha N stati e si fanno $N+1$ transizioni e si ha un loop prima della fine (o alla fine, ma non al di fuori). Quindi come regola può essere considerata valida.

Abbiamo dimostrato che quindi possiamo cambiare il numero di k volte in cui si ripete il ciclo a nostro piacimento

Quindi, in quanto L è regolare, abbiamo determinato N , scelto una stringa w che sia maggiore o uguale in lunghezza di N , e abbiamo dimostrato che tutte le proprietà valgono.

Scritta in modo migliore:

Ipotesi:

$M = \text{DFA minimo che accetta } L$

$N = \text{Numero di stati di } M = |Q_M|$

$z = \text{Stringa appartenente al linguaggio } L$

$m = \text{Lunghezza della stringa, } m \geq N$

Dimostrazione

Prendiamo l'elenco dei $m+1$ stati del percorso che riconoscono la stringa z (stato iniziale + ogni stato per ogni carattere della stringa)

Prendiamo gli $N+1$ stati di questo elenco.

Visto che $m+1 \geq N+1$, allora si passerà sicuramente più volte nello stesso stato del percorso.

Quindi ad un certo punto c'è un loop, dove $q_i = q_j$. Quindi diciamo che gli u sono gli stati fino al loop, poi ci sono gli stati di v nel loop, ed infine c'è w .

Di sicuro uv è minore di N perché i e j sono stati scelti tra i primi $N+1$ stati (con cui attenzione si consumano N archi/simboli). Visto che c'è un loop allora $i \neq j$, allora $v = a_{i+1} \dots a_j$ e quindi $v \geq 1$, in quanto nel caso pessimo $i+1 = j$, quindi ha almeno lunghezza 1 il loop, e negli altri casi ha una lunghezza maggiore

La stringa può essere pompata perché possiamo eseguire il ciclo quante volte ci pare tanto l'automa è deterministico e finirà sempre in uno stato finale quindi riconoscerà la stringa.

56. Come si può utilizzare il pumping lemma (a rovescio) per dimostrare che un linguaggio non è regolare?

Si può usare il pumping lemma a rovescio nel seguente modo:

$$\neg P \Rightarrow \neg L \text{ regolare}$$

Quindi, si fa la negazione del pumping lemma per dimostrare che il risultante non è regolare.

$$\forall N > 0, \exists z \in L, |z| \geq N \Rightarrow$$

$$\begin{aligned} \forall uvw \text{ se } z &= uvw \\ |uv| &\leq N \\ |v| &\geq 1 \end{aligned}$$

allora

$$\exists k \geq 0, \quad uv^k w \notin L$$

$$\Rightarrow L \text{ non è regolare}$$

Esempio di utilizzo:

$$\text{Sia } L = \{a^n b^n \mid n \geq 0\}$$

Fissiamo un N generico ($\forall N > 0$)

Scegliamo $z = a^N b^N$ ($\exists z \in L \text{ con } |z| \geq N$)

Vediamo tutte le sottostringhe possibili tali che valga:

- $|v| \geq 1$
- $|uv| \leq N$
- $z = uvw$

Notiamo che possiamo scegliere uv solo come combinazione di a , per rispettare la seconda premessa. Quindi, abbiamo che v in particolare sarà a^j con $j \geq 1$.

A questo punto noi possiamo pompare una volta questa v , per ottenere:

$$uvvw = a^{N+j} b^N \notin L$$

Quindi abbiamo dimostrato che L non è un linguaggio regolare.

57. Quali sono le proprietà di chiusura dei linguaggi regolari? Quali proprietà si possono decidere (ovvero verificare algoritmicamente)?

La classe dei linguaggi regolari è chiusa per:

1. Unione
2. Concatenamento
3. Stella di Kleene
4. Complementazione
5. Intersezione

DIMOSTRAZIONE:

(1), (2) e (3) sono ovvio. Ad esempio, se L_1 e L_2 sono regolari, allora esistono S_1 e S_2 espressioni regolari tali che $L_1 = \mathcal{L}[S_1]$ e $L_2 = \mathcal{L}[S_2]$. Ma allora $L_1 \cup L_2 = \mathcal{L}[S_1 | S_2]$, cioè $L_1 \cup L_2$ è regolare.

(4) è vero perché, dato un DFA $M = (\Sigma, Q, \delta, q_0, F)$ tale che $L = L[M]$ possiamo costruire il DFA $\bar{M} = (\Sigma, Q, \delta, q_0, Q \setminus F)$ tale che $\bar{L} = L[\bar{M}]$.

Infatti $w \in L[M] \Leftrightarrow w \notin L[\bar{M}]$ e quindi $L[\bar{M}] = \Sigma^* \setminus L[M]$

(5) discende dalla legge di De Morgan

$$L(M_1) \cap L(M_2) = \overline{\overline{L(M_1)} \cup \overline{L(M_2)}}$$

Le proprietà che si possono verificare algoritmicamente sono:

Per i linguaggi regolari, si possono decidere ⁽⁸¹⁾
(cioè verificare algoritmicamente) le seguenti
proprietà:

(M, M_1, M_2 sono DFA)

- $w \in L[M]$? Basta leggere w e vedere se si è
finiti su uno stato finale
- $L[M] = \emptyset$? "Esiste un cammino ^{accettato} dallo
stato iniziale ad uno finale?"
- $L[M] = \Sigma^*$? ($\Leftrightarrow L[\bar{M}] = \emptyset$)
- $L[M_1] \subseteq L[M_2]$? ($\Leftrightarrow L[\bar{M}_2] \cap L[M_1] = \emptyset$)
- $L[M_1] = L[M_2]$? ($\Leftrightarrow L[M_1] \subseteq L[M_2]$
 $\wedge L[M_2] \subseteq L[M_1]$)
o anche: costruisco il DFA
minimo per M_1 e M_2 e
verifico se sono isomorfi
- $L[M]$ è infinito? " $\exists$ una parola $z \in L[M]$
tale che $n \leq z \leq 2n$ dove
 $n = |Q|$?"

Domande 1

09 June 2023 17:39

58. Cosa si intende per analisi sintattica? Cos'è un parser?

Usando la lista di token fatta

Illo scanner, il **parser produce l'albero di derivazione** del programma, riconoscendo se le frasi sono sintatticamente corrette.

- Controlla che (ad esempio)
 - o Le parentesi usate in un'espressione aritmetica sono bilanciate
 - o Che i comandi siano composti secondo le regole grammatiche
 - *if $x = 5$ then then $x := 3$*

Per realizzare un parser dobbiamo studiare:

- **Grammatiche libere da contesto**
- **Automi a pila** (pushdown automata) soprattutto nella versione **determinata** (DPDA)

Il parser è quindi una componente di un compilatore che prende in input una lista di token, prodotti dall'analizzatore lessicale, e genera un albero di derivazione relativo alla grammatica libera che viene usata.

Il parser viene generato a partire da una grammatica libera.

59. **Definizione di automa a pila nondeterministico (PDA).** Definizione di configurazione (o descrizione istantanea), mossa in un passo e mossa in più passi.

Un automa a pila nondeterministico (PDA) è una sestupla definita nel modo seguente:

$(\Sigma, \Gamma, Q, \delta, q_0, F)$ dove:

- $\Sigma :=$ Alfabeto dei simboli in input (che devono essere consumati, la stringa da analizzare usando il PDA)
- $\Gamma :=$ Alfabeto dei simboli della pila
- $Q :=$ Insieme degli stati
- $\delta : Q \times \Sigma \cup \{\epsilon\} \times \Gamma \mapsto \mathcal{P}(Q \times \Gamma^*)$
 - *Quindi, la delta è una funzione di transizione che:*
 - A partire da uno stato, consuma uno o nessun simbolo in input e un simbolo sulla pila
 - E passa ad un qualsiasi numero di stati con un qualsiasi numero di nuovi caratteri sulla pila
- $q_0 \in Q :=$ Lo stato iniziale
- $F \subseteq Q :=$ Stati finali

Una descrizione istantanea di un PDA è una tripla formata da:

$q :=$ lo stato attuale

$\sigma \in \Sigma^* :=$ L'input ancora da consumare

$\gamma \in \Gamma^* :=$ L'insieme di caratteri sulla pila (Quelli più a sinistra sono in cima)

Una mossa in un passo è definita come:

$$\frac{(q', b') \in \delta(q, a, b) \quad a \in \Sigma}{(q, ax, bw) \vdash_{PDA} (q', x, b'w)} \quad oppure \quad \frac{(q', b') \in \delta(q, \varepsilon, b)}{(q, x, bw) \vdash_{PDA} (q', x, b'w)}$$

La mossa in più passi è la chiusura transitiva e riflessiva:

$$\frac{}{(q, w, \beta) \vdash_N^* (q, w, \beta)}, \quad \frac{(q, w, b) \vdash_N^* (q', w', \beta') \quad (q', w', \beta') \vdash_N^* (q'', w'', \beta'')}{(q, w, \beta) \vdash_N^* (q'', w'', \beta')}$$

60. Definizione di linguaggio accettato da un PDA per stato finale o per pila vuota. Sono equivalenti queste due modalità di riconoscimento?

Sia N un PDA $= (\Sigma, \Gamma, Q, \delta, q_0, F)$, riconoscere per stato finale o per pila vuota vuole dire rispettivamente:

$$L[N] = \{w \in \Sigma^* : (q_0, w, \perp) \vdash_N (q, \varepsilon, x) \wedge q \in F \wedge x \in \Gamma^*\}$$

$$P[N] = \{w \in \Sigma^* : (q_0, w, \perp) \vdash_N (q, \varepsilon, \varepsilon)\}$$

Sono equivalenti in quanto è possibile costruire un PDA che riconosce lo stesso linguaggio per pila vuota a partire da uno che lo riconosce per stato finale e viceversa.

Spesso però lo stesso PDA non riconosce lo stesso linguaggio per pila vuota e per stato finale.

61. Mostrare come data una grammatica libera G , sia possibile costruire un PDA P equivalente. È possibile costruire, dato un PDA P , una grammatica equivalente G ? Concludere che la classe dei linguaggi liberi coincide con la classe dei linguaggi riconosciuti da PDA.

È possibile, ma il risultato è molto nondeterministico.

Si può costruire un top down parser del tipo:

Sia la grammatica libera $G = (NT, T, R, S)$

Allora abbiamo un parser del tipo $N = (T, \{q\}, NT \cup T, \delta, q, S, \emptyset)$

Dove vi saranno le funzioni di transizioni del seguente tipo:

Per ogni nonterminali, vi sono le produzioni con una δ del tipo:

$$\delta(q_0, \varepsilon, X) = \{(q_0, Y) : X \in NT, Y \in (NT \cup T)^*, X \rightarrow Y \in R\}$$

Per ogni terminale invece si ha:

$$\delta(q_0, a, a) = \{(q_0, \varepsilon)\} \quad \forall a \in T$$

In questo modo si crea un PDA N tal che $L = L(G) = P[N]$.

Si può anche fare il contrario, oppure partendo da PDA P si può costruire una grammatica G . Vengono forniti due lemmi per la dimostrazione, che non abbiamo approfondito, ovvero che ogni

PDA N può essere simulato in un PDA N' con un solo stato, aumentando opportunamente i simboli sulla pila e che ogni PDA con un solo stato ha un equivalente grammatica libera.

Pertanto questo ci dice che la classe dei linguaggi liberi coincide con quella dei PDA in quanto da un formalismo possiamo ridurci a trovare l'altro e viceversa.

62. Quali sono le proprietà di chiusura dei linguaggi liberi? L'intersezione di un linguaggio libero con un linguaggio regolare è un linguaggio libero?

Le proprietà di chiusura dei linguaggi liberi sono:

- Unione: aggiungendo $S \rightarrow S1 \mid S2$
- Concatenazione: aggiungendo $S \rightarrow S1S2$
- Ripetizione: aggiungendo $S \rightarrow SS_1 \mid \varepsilon$

Non sono chiusi per intersezione e pertanto neanche per complementazione in quanto si potrebbe sennò usare De Morgan sull'unione.

L'intersezione con un linguaggio libero e uno regolare dà sempre un linguaggio libero.

63. Intestazione e dimostrazione del "Pumping Theorem".

L'intestazione del pumping theorem è:

G è una grammatica libera $\Rightarrow$
 $\exists N > 0, \quad \forall z \in L(G), \quad |z| \geq N, \quad \exists u, v, w, x, y \quad \text{tale che:}$

- $z = uvwxy$
- $|vwx| \leq N$
- $|vx| \geq 1$
- $\forall k \in \mathbb{N}, \quad uv^kwx^ky \in L(G)$

Dimostriamo ora il pumping theorem.

Sia $G = (NT, T, R, S)$ una grammatica libera tale che $L = L(G)$.

Sia b il massimo fattore di ramificazione di un albero di derivazione (ovvero il massimo numero di simboli che compaiono nella parte destra di una produzione in R .)

$b = \max\{|a| \mid A \rightarrow a \in R\}$
($b \geq 2$ altrimenti la grammatica sarebbe banale)

Un albero di derivazione di altezza h (con 0 la radice) è fattore di ramificazione b , ha al più b^h foglie.

(Immaginiamo di avere per ogni produzione il fattore di ramificazione massimo. Immaginiamo questo fattore di ramificazione massimo sia due. Allora ogni livello di altezza genera due rami. Ad altezza 0 si ha 1 nodo, altezza 1 si hanno 2 nodi, altezza 2 si hanno 4 nodi, altezza 3 si hanno 8 nodi, etc.)

Fissiamo $N = b^{|NT|+1}$ (Quindi $N > b^{|NT|}$ dato che $b \geq 2$)

Allora ogni albero di derivazione per z , con $|z| \geq N$, deve avere altezza almeno $|NT| + 1$.

Prendiamo una qualunque $z \in L$, con $|z| \geq N$.

Consideriamo il suo albero di derivazione (se ne possiede più di uno, perché G è ambigua,

prendiamo quello con minor numero di nodi).

Dunque:

$|z| \geq N \Rightarrow$ *Albero con altezza $\geq |NT| + 1$*
 $\Rightarrow \exists$ cammino da radice S ad una foglia con almeno $|NT| + 2$ nodi
 $\Rightarrow$ Quel cammino attraverso $|NT| + 1$ nodi intermedi etichettati con nonterminali
(La foglia è etichettata da un terminale)
 $\Rightarrow$ Almeno un nonterminale si ripete in quel cammino

Allora si può dividere $S \Rightarrow^* z$ in:

$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uvwxy$

La parte dove si passa da uAy ad $uvAxy$ si può ripetere più volte.
Per esempio:

$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uv^2Ax^2y \Rightarrow^* uv^2wx^2y$

Possiamo anche non ripeterla, in quanto vediamo che per la produzione di A si può passare sia, in un numero finito di passaggi, a vAx che a w .

Quindi bisogna verificare se i vincoli sono veri:

- $|vx| \geq 1$: Vero, in quanto se non lo fosse, pertanto sia v che x fossero ε allora l'albero per $k = 0$ genererebbe ancora z ma con meno nodi, in quanto lo genera per $k=1$ (e per $k \in \mathbb{N}$). Quindi questo contraddice l'ipotesi di aver scelto il più piccolo albero.
- $|vwx| \leq N$: Vero, perché per passare da A a vwx usiamo sicuramente un numero minore o uguale a $|NT| + 1$ nodi, in quanto la profondità massima è $|NT| + 2$ e c'è un terminale. Quindi, in quanto il massimo numero di produzioni di ogni NT è b , allora sicuramente la stringa non potrà essere più lunga di $b^{|NT|+1}$ che è proprio la nostra N . Quindi anche se ci fossero $|NT| + 1$ nodi, caso peggiore, allora la stringa non potrebbe essere più lunga di N .

64. Come si può utilizzare tale teorema (a rovescio) per dimostrare che un linguaggio non è libero?

Per dimostrare $\neg L$ libero, ci possiamo ridurre a dimostrare che $\neg Pumping\ theorem$, ovvero:

L libero $\Rightarrow$ Proprietà
 $\neg Proprietà \Rightarrow \neg L$ libero

Scriviamo quindi cosa è la proprietà da dimostrare negata:

$\forall N > 0, \quad \exists z \in L(G), \quad |z| \geq N, \quad \forall u, v, w, x, y \quad se:$
- $z = uvwxy$
- $|vwx| \leq N$
- $|vx| \geq 1$

Allora $\exists k \in \mathbb{N}, \quad uv^kwx^ky \notin L(G)$

Facciamo un esempio, dimostriamo che $L = \{a^n b^n c^n : n \geq 0\}$ non è un linguaggio libero.

Fissiamo una N generica. Scegliamo una stringa z del tipo:

$$z = a^N b^N c^N. \text{ Abbiamo che } |z| \geq N \text{ e che } z \in L(G)$$

Dimostriamo che, indipendentemente da come spezzettiamo la stringa, c'è sempre una k per cui questo spezzettamento non è corretto.

Dobbiamo sceglierli in tal modo che $|vwx| \leq N$ e $|vx| \geq 1$

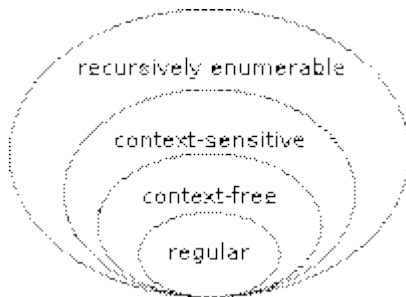
Notiamo che quello che dovremmo fare sarebbe aumentare di uno stesso numero il numero di a , b e c . Notiamo che però:

- Se la v contenesse solo a e la x contenesse solo b , si sbilancia
- Se la v contenesse solo b e la x contenesse solo c , si sbilancia
- Se sia v che c contenessero la stessa lettera, si sbilancia
- In altre combinazioni di v e x che sono formati da porzioni di lettere adiacenti nella stringa, si sbilancia
- E per qualsiasi altra combinazione, $|uwx|$ non sarebbe minore o uguale a N .

Quindi $a^N b^N c^N$ non è libero da contesto

65. Classificazione di Chomsky delle grammatiche e dei linguaggi. Definizione di grammatica dipendente da contesto e di grammatica monotona. Quale tipo di automi corrisponde ad ogni classe?

La classificazione di Chomsky permette di mostrare l'appartenenza delle varie grammatiche e di quale sia un sottoinsieme di un'altra.



Vengono divise in tipologie, ecco come vengono trascritte da Wikipedia:

Gram.	Linguaggio	Automa	Regole di produzione	Esempi
Tipo 0	Ricorsivamente numerabile	Macchina di Turing	$\gamma \rightarrow \alpha$ (γ non - empty)	$L = \{w \mid w \text{ descrive una Turing Machine che termina}\}$
Tipo 1	Dipendente da contesto	Macchina di Turing linearmente limitata	$\alpha A \beta \rightarrow \alpha \gamma \beta$	$L = \{a^n b^n c^n \mid n > 0\}$
Tipo 2	Libero da contesto	PDA	$A \rightarrow \alpha$	$L = \{a^n b^n \mid n > 0\}$
Tipo 3	Regolare	NFA/DFA	$A \rightarrow a$	$L = \{a^n \mid n \geq 0\}$

Una grammatica dipendente da contesto è una grammatica in cui le regole di produzione consentono di sostituire una variabile non terminale con una stringa di simboli che può includere sia simboli terminali che simboli non terminali. Tuttavia, ciò non impone restrizioni sulla lunghezza della stringa sostituiva rispetto alla stringa originale.

In altre parole, la lunghezza della stringa può aumentare, diminuire, o rimanere la stessa durante la sostituzione delle regole di produzione.

Una grammatica monotona è una grammatica dipendente da contesto in cui tutte le regole di produzione sostituiscono una variabile non terminale con una stringa di simboli che ha la stessa o maggiore lunghezza della stringa originale. In altre parole, una grammatica monotona non diminuisce la lunghezza delle stringhe durante la sostituzione delle regole di produzione. Pertanto, ogni sostituzione della variabile non terminale non può abbreviare la lunghezza della stringa.

Tutte le grammatiche monotone sono grammatiche dipendenti da contesto, ma non tutte le grammatiche dipendenti da contesto sono monotone. La monotonia è una restrizione aggiuntiva imposta alle regole di produzione che garantisce che la lunghezza delle stringhe non diminuisca durante le sostituzioni.

I linguaggi monotoni e dipendenti sono equivalenti, le grammatiche dipendenti sono invece contenute dalle monotone.

Leggendo su internet, le grammatiche dipendenti sono definite in quel modo, ma altri autori si riferiscono alle grammatiche monotone quando scrivono grammatiche dipendenti da contesto. In quanto le prime sono uguali alle seconde, allora si possono scrivere grammatiche di uno o di un altro tipo e sono equivalenti, pertanto non è davvero un problema. (Questo se sono "non-contracting", ovvero se non hanno produzioni di dimensione minore, a parte per S che può avere una produzione vuota)

SCRITTE DALLE SLIDE:

- Grammatiche regolari
 $A \rightarrow aB, \quad A \rightarrow a, \quad S \rightarrow \varepsilon$
- Grammatiche libere da contesto
 $A \rightarrow \gamma, \quad \gamma \in (NT \cup T \setminus \{S\})^+, \quad S \rightarrow \varepsilon$
- Grammatiche dipendenti da contesto
 $\gamma A \delta \rightarrow \gamma w \delta, \quad \gamma, \delta \in (NT \cup T \setminus \{S\})^*, \quad w \in (NT \cup T \setminus \{S\})^+, \quad S \rightarrow \varepsilon$
- Grammatiche monotone
 $\gamma \rightarrow \delta, \quad |\gamma| \leq |\delta|$
- Grammatiche generali
 $\gamma \rightarrow \delta, \quad \text{senza vincoli}$

66. Definizione di DPDA (automa a pila deterministico) e di linguaggio libero deterministico.

Un DPDA è N è una 7-upla rappresentata da:

$(\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ dove ognuno è definito nel seguente modo:

- Σ := Insieme dei simboli in input
- Q := Insieme degli stati dell'automa
- Γ := Insieme dei simboli sulla pila
- δ := *funzione di transizione che fa*:
$$\delta : Q \times (\Sigma \times \{\varepsilon\}) \times \Gamma \mapsto Q \times (\Gamma \cup \varepsilon)$$

Tale che, $\forall q \in Q, \forall A \in \Gamma, \forall \sigma \in \Sigma \cup \{\varepsilon\}$:

$$|\delta(q, \varepsilon, A)| \neq 0 \Rightarrow |\delta(q, \sigma, A)| = 0, \quad \forall \sigma \in \Sigma$$
$$|\delta(q, a, A)| \leq 1, \quad \forall A \in \Gamma, \quad \forall a \in \Sigma \cup \{\varepsilon\}$$

(Per ricordarmelo a parole: per ogni cosa c'è al massimo una mossa. Inoltre, se c'è una mossa epsilon per un elemento della pila, allora non vi sono mosse che consumano lo stesso elemento della pila e non sono epsilon).

- $q_0 \in Q$:= *stato iniziale*
- $\perp \in \Gamma$:= *elemento iniziale della pila*
- $F \subseteq Q$:= *Stati finali*

Un linguaggio libero deterministico è un linguaggio riconosciuto da un DPDA per stato finale.

67. La classe dei linguaggi liberi deterministici è strettamente inclusa in quella dei linguaggi liberi? Contiene strettamente la classe dei linguaggi regolari?

Sì e sì. Per la seconda, si può costruire un DPDA a partire dal DFA senza mai andare a toccare la pila.

68. Che cosa dice la prefix property e perché è interessante per i DPDA?

La prefix property ci dice che nel linguaggio L non esistono $x, y \in L$ tale che x è prefisso di y , ovvero $y = xz$.

È importante in quanto:

L gode della prefix property $\Leftrightarrow L$ riconosciuto da un DPDA per pila vuota

69. Usando un endmarker \$, si può riconoscere un linguaggio libero deterministico che non gode della prefix property anche per pila vuota? Come?

Sì, aggiungendo \$ alla fine di ogni stringa del linguaggio libero deterministico non ci saranno mai due stringhe che siano una il prefisso dell'altra, in quanto ogni stringa sarà "univoca" per l'avere \$ solo come finale, e mai in mezzo.

Pertanto potrà essere riconosciuto da un DPDA per pila vuota in quanto godrà della prefix property.

70. Un linguaggio libero deterministico è ambiguo?

No, non sono mai ambigui perché sappiamo che se L è libero deterministico, allora L è generabile da una G libera non ambigua.

71. Proprietà di chiusura dei linguaggi liberi deterministici: chiusi per complementazione,

ma non per unione o intersezione.

I linguaggi sono chiusi per complementazione. In particolare, per la dimostrazione, si deve anche completare δ , aumentando N con più stati, eventualmente finali o non finali. A questo punto si scambiano finali e non finali.

Non sono chiusi per intersezione e neanche per unione in quanto si potrebbe usare De Morgan.

Esempio di intersezione per cui non sono chiusi:

$$L_1 = \{a^n b^n c^m | n, m \geq 0\} \text{ è lib det}$$

$$L_2 = \{a^n b^m c^m | n, m \geq 0\} \text{ è lib det}$$

Ma

$$L_1 \cap L_2 = \{a^n b^n c^n | n \geq 0\} \text{ non è lib}$$

72. Da cosa si parte per costruire un analizzatore sintattico (ovvero parser)? Da un'espressione regolare? Da una grammatica libera? Da un PDA?

A partire da una grammatica libera si andrà a costruire un PDA che riconosce le sue produzioni. Esistono PDA diversi, ma sono sempre PDA, per il top-down e il bottom-up parsing.

73. Cosa prende in input e cosa produce in output un parser?

Un parser prende in input una lista di token, provenienti dallo scanner, e in output ritorna un albero di derivazione.

74. Che differenza c'è tra un parser nondeterministico ed uno deterministico?

Un parser nondeterministico compie scelte che, per quanto probabili, non è completamente sicuro della loro correttezza, e pertanto fa dei "tentativi": nel caso sbagliasse, dovrebbe tornare indietro e provare in un altro modo, fino a quando prova tutti i casi oppure ne trova uno che funziona.

Un parser deterministico è completamente sicuro delle sue scelte, ognuna di esse è finale. Nel caso di errore, c'è sicuramente stato e non ha bisogno di tornare indietro per provare altre "strade".

75. Quali sono le due tecniche essenziali per costruire parser?

Sono le tecniche bottom-up e top-down.

La tecnica top-down parte dall'avere sulla pila il simbolo iniziale "S" e poi espanderlo usando le produzioni. Quando sulla pila vi sono dei terminali che coincidono con la stringa che si legge, essi vengono tolti dalla pila. Quando si incontrano di nuovo nonterminali, essi si espandono.

Vi è quindi la parte di *expand* e la parte di *consume*.

Si parte da S sulla pila e si espande man mano, ricostruendo la stringa.

Definizione : Data una grammatica libera $G = (NT, T, S, R)$, costruiamo il PDA $M = (T, \{q\}, T \cup NT, \delta, q, S, \emptyset)$ che riconosce per pila vuota (infatti non ha stati finali) e dove δ è definita così:

$$\begin{array}{ll} (q, \beta) \in \delta(q, \epsilon, A) & \text{se } A \rightarrow \beta \in R \quad (\text{espandi}) \\ (q, \epsilon) \in \delta(q, a, A) & \forall a \in T \quad (\text{consuma}) \end{array}$$

tale che $L(G) = P[M]$.

La tecnica bottom-up parte dall'avere sulla pila un simbolo iniziale "Z", che non fa parte dei terminali o dei nonterminali. Man mano che legge la stringa, esegue delle shift, inserendola dentro la stack. Quando vi è una stringa che fa parte di una produzione nella stack, allora viene fatto un reduce e si trova il nonterminale della produzione. Si continua questo procedimento fino al trovare il simbolo finale del dollaro, con il quale si fa un accept nel caso sia tutto corretto.

Vi è quindi *shift*, *reduce* ed *accept*.

Si parte da una pila vuota e si inseriscono i caratteri letti, ricostruendo la S

Definizione : Data una grammatica libera $G = (NT, T, R, S)$ costruiamo un PDA M che riconosce $L(G)\$$ con $M = (T, \{q\}, T \cup NT \cup \{z\}, \delta, q, z, \emptyset)$, dove la funzione δ è così definita:

$$\begin{array}{ll} (q, aX) \in \delta(q, a, X) & \forall a \in T, \forall x \in T \cup NT \cup \{z\} \quad (\text{shift}) \\ (q, A) \in \delta(q, \epsilon, \alpha^R) & \text{se } A \rightarrow \alpha \in R \quad (\text{reduce}) \\ (q, \epsilon) \in \delta(q, \$, SZ) & (\text{accept}) \end{array}$$

76. Le tecniche top-down e bottom-up in che cosa differiscono?

Innanzitutto, l'albero di derivazione che generano:

- Top down genera una derivazione leftmost (albero di derivazione generato dall'espansione del nonterminale più a sinistra)
- Bottom up genera una derivazione rightmost (albero di derivazione generato dall'espansione del nonterminale più a destra)

Cambiano anche il tipo di grammatiche che accettano e quali determinano un più forte nondeterminismo e quali meno.

Inoltre, bottom up parte dalla stringa e genera la S, mentre top down parte da S e genera la stringa.

77. Quali tipi di grammatiche non sono adatte al top-down parsing?

Le grammatiche non adatte al top-down parsing sono quelle:

- Left recursive
- Ovvero il terminale si espande in lui stesso seguito da un altro

Le grammatiche non adatte al bottom-up parser sono quelle:

- Con produzioni epsilon
- In quanto sono sempre applicabili e rendono il parser fortemente nondeterministico

78. Cosa sono le produzioni epsilon e cosa sono i simboli non-terminali annullabili?

Le produzioni epsilon sono qualsiasi tipo di produzione:

$$A \rightarrow \epsilon \in R$$

I simboli annullabili sono non terminali tali che $A \Rightarrow^+ \epsilon$. Li indichiamo come $N(G)$, data una

grammatica G. In particolare, definiamo $N(G)$ in modo ricorsivo:

$$N_0(G) = \{A \in NT \mid A \rightarrow \varepsilon\}$$

$$N_i(G) = N_{i-1}(G) \cup \{A \in NT \mid A \rightarrow C_1 \dots C_n \in R \wedge C_1, \dots, C_n \in N_{i-1}(G)\}$$

$\exists i_c$ tale che $N_{i_c}(G) = N_{i_c+1}(G)$ in quanto NT è finito

$$N_{i_c}(G) = N(G)$$

79. Come si può trasformare una grammatica G che contiene produzioni epsilon in una grammatica G' che non ne contiene, preservando il linguaggio a meno di epsilon?

Una volta calcolato $N(G)$ per $G=(NT,T,S,R)$, costruiamo la grammatica $G'=(NT,T,S,R')$ dove modifichiamo le produzioni.

$\forall R$ tale che $A \rightarrow \alpha$ con $\alpha \neq \varepsilon$

In cui occorrono simboli annullabili $z_1, \dots, z_k$

Mettiamo in R' tutte le produzioni del tipo:

$A \rightarrow \alpha'$, dove α' si ottiene da α cancellando tutti i possibili sottoinsiemi di $z_1, \dots, z_k$ (incluso $\emptyset$) **ad eccezione** del caso in cui α' risulta in ε .

- In G' non mettiamo produzioni $A \rightarrow \varepsilon \in R$
- In G' non introduciamo mai produzioni del tipo $A \rightarrow \varepsilon$

Quindi:

- Ci troviamo $N(G)$
- Modifichiamo ogni produzione dove appare un elemento di $N(G)$:
 - Consideriamo gli elementi dell'insieme potenza generato dal sottoinsieme di $N(G)$ di non terminali che compaiono nella produzione che si sta correggendo.
 - Per ognuno di questi elementi, rimuoviamo il non terminale che appare nella produzione.
 - Consideriamo le produzioni così ottenute al posto della originale, eliminando quelle che sarebbero produzioni ε

Esempio:

Esempio:

(23)

$$G = \begin{cases} S \rightarrow AB \\ A \rightarrow aAA \mid \epsilon \\ B \rightarrow bBB \mid \epsilon \end{cases} \quad \begin{aligned} N_0(G) &= \{A, B\} \\ N_1(G) &= \{A, B, S\} \\ &= N(G) \end{aligned}$$

Consideriamo $S \rightarrow AB \in R$. Dobbiamo considerare

4 casi:

$$\begin{aligned} \emptyset &\Rightarrow S \rightarrow AB & S \rightarrow AB \mid A \mid B \\ \{B\} &\Rightarrow S \rightarrow A \\ \{A\} &\Rightarrow S \rightarrow B \\ \{A, B\} &\Rightarrow S \rightarrow \epsilon \quad \text{non la mettiamo in } R' \end{aligned}$$

Ora consideriamo $A \rightarrow aAA \in R$. Dobbiamo considerare

4 casi:

$$\begin{aligned} \emptyset &\Rightarrow A \rightarrow aAA \\ \{A\} &\Rightarrow A \rightarrow aA \\ \{A\} &\Rightarrow A \rightarrow aA \\ \{A, A\} &\Rightarrow A \rightarrow a \end{aligned} \quad \begin{aligned} & \\ & \} \text{ duplicate} \\ & \Rightarrow A \rightarrow aAA \mid aA \mid a \end{aligned}$$

Ora consideriamo $B \rightarrow bBB$, e nello stesso modo
di prima, otteniamo $B \rightarrow bBB \mid bB \mid b$

Le due ϵ -produzioni $A \rightarrow \epsilon$ e $B \rightarrow \epsilon$ sono ignorate.

$$\Rightarrow G' = \begin{cases} S \rightarrow AB \mid A \mid B \\ A \rightarrow aAA \mid aA \mid a \\ B \rightarrow bBB \mid bB \mid b \end{cases}$$

N.B. $\epsilon \in L(G) \quad S \Rightarrow AB \Rightarrow B \Rightarrow \epsilon$
 $\epsilon \notin L(G')$

$$G'' = \begin{cases} S' \rightarrow \epsilon \mid S \\ S \text{ come in } G' \end{cases} \quad \text{e tale che } L(G) = L(G'')$$

80. Cosa sono le produzioni unitarie e cosa sono le coppie unitarie?

Una produzione unitaria è una produzione del tipo:

$$A \rightarrow B \in R, \quad \text{tale che } A \in NT \text{ e } B \in NT$$

Una coppia unitaria è una coppia del tipo (A, B) , con $A \in NT$ e $B \in NT$ tale che:

$$A \Rightarrow^* B, \quad \text{con solo produzioni unitarie}$$

Definiamo quindi l'insieme delle produzioni unitarie $U(G)$ della grammatica G nel seguente modo:

$$\begin{aligned} U_0(G) &= \{(A, A) \mid A \in NT\} \\ U_i(G) &= U_{i-1}(G) \cup \{(A, B) \mid A, B, C \in NT \wedge (A, C) \in U_{i-1}(G) \wedge C \rightarrow B \in R\} \end{aligned}$$

Esiste una i_C tale che:

$$U_{i_c}(G) = U_{i_c+1}(G)$$

E quindi:

$$U_{i_c}(G) = U(G)$$

81. Come si può trasformare una grammatica G che contiene produzioni unitarie in una equivalente G' che non ne contiene?

Creiamo una G' tale che ogni sua produzione non presenta le produzioni unitarie. Inoltre, per ogni coppia unitaria, andiamo ad assegnare le produzioni del secondo nonterminale a quelle del primo.

82. Data una grammatica G, quali sono i suoi simboli utili? Quali sono i suoi simboli generatori? Quali i suoi simboli raggiungibili?

I suoi simboli utili sono quelli che sono sia generatori che raggiungibili. I suoi simboli generatori sono quei simboli tali che:

$$a \Rightarrow^* b \text{ tale che } b \in T^*, \quad a \in NT \cup T$$

I simboli raggiungibili sono invece quei simboli tali che:

$$S \Rightarrow^* aXy \text{ tale che } S \text{ è simbolo iniziale, } a, y \in (T \cup NT)^*, \quad X \in (NT \cup T)$$

83. Come si calcolano i generatori? Come si calcolano i raggiungibili?

Definiamo, come per le altre, i generatori e i raggiungibili in modo induttivo:

Generatori:

1. $G_0(G) = T$, se $a \in T, a \Rightarrow^* a$ quindi tutti i terminali sono generatori
2. $G_{i+1}(G) = G_i(G) \cup \{B \in NT \mid B \rightarrow c_1 \dots c_k \in R \wedge c_1, \dots, c_k \in G_i(G)\}$

Raggiungibili:

1. $R_0(G) = \{S\}$
2. $R_i(G) = R_{i-1}(G) \cup \bigcup_{\substack{B \in R_{i-1}(G) \\ B \rightarrow c_1 \dots c_n \in R}} \{c_1, \dots, c_n\}$

84. In che modo si eliminano i simboli inutili di una grammatica? È importante l'ordine delle operazioni da svolgere?

Vanno prima rimossi i simboli generatori e poi quelli raggiungibili. È importante l'ordine da svolgere in quanto se si rimuovessero prima i raggiungibili e poi i generatori, rimuovendo questi ultimi potremmo avere dei nuovi simboli non raggiungibili, quindi non avremmo concluso la nostra eliminazione.

Per eliminare questi simboli:

- 1- Si eliminano tutte le produzioni che contengono simboli non generatori
- 2- Si eliminano tutte le produzioni che contengono simboli non raggiungibili

85. Quando si dice che una grammatica è ricorsiva sinistra? Come si elimina la ricorsione immediata? E quella non immediata? Perché serve eliminare la ricorsione sinistra?

Si definisce ricorsione sinistra:

$$A \Rightarrow^+ Aa, \quad A \in NT, \quad a \in (T \cup NT)^*$$

Per eliminare la ricorsione sinistra immediata bisogna fare:

Sia una ricorsione sinistra immediata definita come:

$$A \rightarrow Aa_1 \mid \dots \mid Aa_n \mid b_1 \mid \dots \mid b_k$$

Allora possiamo creare un equivalente senza ricorsione sinistra immediata usando un nuovo simbolo A' in questo modo:

$$A \rightarrow b_1A' \mid \dots \mid b_kA'$$

$$A' \rightarrow a_1A' \mid \dots \mid a_nA' \mid \varepsilon$$

Per eliminare la ricorsione sinistra non immediata:

impostiamo un ordine nei non terminali: $\{A_1, \dots, A_n\}$

per i da 1 a n {

per j da 1 a $i - 1$ {

Modificare tutte le produzioni del tipo $A_i \rightarrow A_j a$ in:

$A_i \rightarrow ba$, dove b sono tutte le produzioni $A_j \rightarrow b$

}

Rimuovi ricorsione immediata per A_i

}

Serve eliminare la ricorsione sinistra in quanto non è possibile fare parsing top-down usando la ricorsione sinistra. Espandendo A con A di nuovo, non legge mai il simbolo che dovrebbe leggere in input come primo elemento, pertanto non consuma mai ma fa solo espansione.

86. Cosa vuol dire fattorizzare a sinistra una grammatica? Perché serve fattorizzare?

Significare raccogliere i terminali comuni che si trovano all'inizio delle produzioni, sostituendo quello che segue con un nuovo nonterminale che ripete la produzione rimanente.

Serve fattorizzare in quanto elimina molto nondeterminismo per i parser top down. Riduce di molto il look ahead.

87. Cos'è un parser a discesa ricorsiva?

È un parser molto nondeterministico che è definito dalla seguente funzione:

function $A()$ {

Scegli una delle k produzioni di A : h tra 1 e k

$A \rightarrow x_1^h \dots x_n^h$

```

 $\forall x_i^h \{$ 
    if  $x_i^h \in NT$  allora  $x_i^h( )$ 
    else if  $x_i^h$  è simbolo in input allora rimuovilo dagli input
    else Fail()
        backtracking
        fino a quando
        va bene
     $\}$ 
 $\}$ 

```

Si inizia chiamando $S()$.

Domande 2

mercoledì 14 giugno 2023 10:47

88. Definizione di $First(\alpha)$ e di $Follow(A)$.

$First(\alpha)$ sono i $T \cup NT \cup \{\varepsilon\}$ che vengono definiti nel seguente modo:

$$\begin{aligned}\alpha \Rightarrow^* a\beta &\Rightarrow a \in First(\alpha), & \text{per qualche } \beta \in (NT \cup T)^* \\ \alpha \Rightarrow^* \varepsilon &\Rightarrow \varepsilon \in First(\alpha)\end{aligned}$$

I $Follow(A)$ sono definiti nel seguente modo:

$$\begin{aligned}b \in T \wedge S \Rightarrow^* xAby &\Rightarrow b \in Follow(A), & \text{per qualche } x, y \in (T \cup NT)^* \\ S \Rightarrow^* xA &\Rightarrow \$ \in Follow(A)\end{aligned}$$

89. Algoritmi per calcolare $First(\alpha)$ e $Follow(A)$

Per calcolare i $First(\alpha)$:

```
 $\forall t \in T, \quad First(t) = \{t\}$   
 $\forall nt \in NT, \quad First(nt) = \emptyset$   
determinare  $N(G)$  (simboli annullabili di  $G$ )  
while avvengono modifiche, do:  
    per ogni produzione  $nt \rightarrow y_1 \dots y_k$   
        da  $i = 1$  a  $k$ :  
            se  $y_1, \dots, y_{i-1} \in N(G)$   
                allora  $First(nt) = First(nt) \cup First(y_i) \setminus \{\varepsilon\}$   
se  $nt \rightarrow \varepsilon \in R$ , allora  $First(nt) = First(nt) \cup \{\varepsilon\}$ 
```

Per calcolare i $Follow(A)$

```
 $\forall nt \in NT, \quad Follow(nt) = \emptyset$   
 $Follow(S) = \{\$ \}$   
 $\forall A \rightarrow xB\gamma \in R \quad \text{add } First(\gamma) \setminus \{\varepsilon\} \text{ to } Follow(B)$   
 $\forall A \rightarrow xB \in R \quad \text{o} \quad \forall A \rightarrow xB\gamma \in R \wedge \varepsilon \in First(\gamma) \quad \text{add } Follow(A) \text{ to } Follow(B)$ 
```

90. Come è fatta e come si riempie una tabella di parsing LL(1)?

Una tabella di parsing LL(1) è una tabella che ha:

Come colonne, i simboli terminali e il dollaro
Come righe, i simboli non terminali

Come elementi della tabella le produzioni.

In particolare, nella posizione $M[X, i_c]$ ha la produzione:

$$X \rightarrow Y \in R \quad \text{tale che} \quad i_c \in First(Y)$$

Inoltre, se $\varepsilon \in First(Y)$, allora:

$X \rightarrow Y \in R$ viene inserito in $M[X, t]$, $\forall t \in Follow(X)$

91. Quando una grammatica G si dice di classe LL(1)? Quali sono le condizioni necessarie e sufficienti per G affinché sia di classe LL(1)?

Una grammatica si dice di classe LL(1) quando si può creare una tabella di parsing LL(1) che non ha più di una produzione in ogni sua cella.

Le condizioni sufficienti e necessarie affinché G sia di classe LL(1) sono:

Si considerino coppie di produzioni a partire dallo stesso non terminale, ovvero: $A \rightarrow a|b$. G è di classe LL(1) $\Leftrightarrow$ valgono entrambe le seguenti affermazioni:

- $First(a) \cap First(b) = \emptyset$
- $\varepsilon \in First(a) \Rightarrow Follow(a) \cap First(b) = \emptyset$
- $\varepsilon \in First(b) \Rightarrow Follow(b) \cap First(a) = \emptyset$

92. Perché un parser di questo tipo è chiamato LL?

Un parser di questo tipo è chiamato LL è in quanto:

- L := Legge la stringa in input left to right
- L := Ritorna l'albero di derivazione della derivazione sinistra della stringa in input

93. Come funziona il parser LL(1) con una pila?

Inizializza una pila con S\$.

Inizializziamo il simbolo iniziale della pila con X

Inizializziamo il simbolo iniziale in lettura con i_c

Fino a quando ($X \neq \$$)

Se $X \in NT$ allora expand:

pop di X dalla pila

push della produzione di $M[X, i_c]$ sulla pila

Se cella vuota, allora errore

set X to top della pila

Se $X = i_c$ allora consume

rimuovi X dalla pila

consuma il simbolo in input i_c

imposta X to top della pila e i_c to top della stringa in lettura

Altrimenti

Errore

Se $i_c \neq \$$, errore

94. È vero che ogni linguaggio regolare è pure LL(1)?

Sì, ogni linguaggio regolare è LL(1) in quanto si può costruire un DFA da esso, dal quale si può costruire una grammatica tale che i NT siano gli stati del DFA. Il DFA è deterministico quindi anche le transizioni lo saranno, pertanto non ci possono essere più di 1 transizione (1 produzione) con lo stesso nonterminale, pertanto l'intersezione tra i first è vuota, per ogni coppia di produzioni.

95. Come sono definiti $First_k(\alpha)$ e di $Follow_k(A)$ per $k \geq 2$?

$$First_k(\alpha) = \{x \in T^* \mid |x| \leq k \wedge \alpha \Rightarrow^* x\beta, \text{ per qualche } \beta \in (T \cup NT)^*\} \cup \{\epsilon \text{ se } \alpha \Rightarrow \epsilon\}$$

$$Follow_k(A)$$

$$= \{x \in T^* \mid |x| \leq k \wedge S \Rightarrow^* dAxy, \text{ per qualche } d, \gamma \in (T \cup NT)^*\} \cup \{\$ \text{ se } S \Rightarrow^* dA \text{ per qualche } d \in (T \cup NT)^*\}$$

96. Come si definisce una grammatica LL(k)? Ed un linguaggio LL(k)? Come si relazionano tra di loro?

Una grammatica LL(k) è una grammatica per la quale si può creare una tabella di parsing LL(k) tale che non vi siano più di una produzione per ogni cella.

Un linguaggio LL(k) è un linguaggio per cui è possibile avere una grammatica LL(k).

Ci possono essere grammatiche LL(k) per cui il linguaggio in realtà è LL(x) con $x < k$.

97. Che relazione esiste tra grammatiche LL(k) e grammatiche ambigue? E con le grammatiche ricorsive sinistre?

Una grammatica ambigua o una grammatica ricorsiva sinistra non è LL(k) per nessun k.

98. Esistono linguaggi liberi che non sono LL(k) per nessun k? Ed esistono linguaggi liberi deterministici che non sono LL(k) per nessun k?

Sì, possono esserci sia linguaggi liberi che linguaggi liberi deterministici che non sono LL(k) per nessun k.

Però se G è LL(k) per qualche k, allora L(G) è libero deterministico e non è ambiguo.

99. Cos'è un parser bottom-up (o shift-reduce)? Qual è il suo input e il suo output? Perché sono chiamati parser LR?

Un parser bottom-up è un parser che ricostruisce la S a partire dalla stringa, usando in modo inverso le produzioni. Compie tre operazioni

shift := legge un simbolo della stringa in input e lo mette nella pila dei simboli letti

reduce := se ha una produzione $A \rightarrow a$ e sulla pila dei simboli letti ha a^R , allora applica una reduce e sostituisce a con A sulla pila.

accept := se legge \$ ed ha il simbolo iniziale, sia esso S, allora accetta la stringa.

Si può andare anche a definire un parser bottom-up usando un PDA non deterministico nel seguente modo:

$$N = (\Sigma, Q, \Gamma, \delta, q_0, \perp, F) = (T \cup \{\$, \{q\}, T \cup NT \cup \{\$, \delta, q, \$, \emptyset)$$

Ed esso riconosce, per pila vuota, attraverso una funzione di transizione definita nel seguente

modo:

$$\begin{aligned}\delta(q, a, A) &= (q, aA), & \forall a \in T & - \text{operazione di shift} \\ \delta(q, \varepsilon, a^R) &= (q, Y), & \forall Y \rightarrow a & - \text{operazione di reduce} \\ \delta(q, \$, \$) &= (q, \varepsilon), & & \text{operazione di accept}\end{aligned}$$

Notiamo che esso ha un solo stato e che effettua esattamente le operazioni descritte, anche se in questa definizione lo fa in modo molto nondeterministico.

Il suo input è una lista di token, mentre il suo output è un albero di derivazione rightmost.

Sono chiamati LR in quanto leggono da sinistra a destra l'input (L) e creano l'albero di derivazione rightmost.

100. Che tipo di conflitti si possono presentare in un parser del genere? Quando si presenta un conflitto (shift/reduce o reduce/reduce), quale azione bisogna scegliere?

Si possono presentare due tipi di conflitti, ovvero:

- Shift/reduce
 - Se per esempio uno stato di un LR(0) presenta due item del tipo:
 - $S \rightarrow a.$; $S \rightarrow a.x$
 - In questo caso, vi è un conflitto nella tabella di parsing in quanto la stessa cella presenta due opzioni.
- Reduce/reduce
 - Se non si sa quale di due reduce fare, per esempio se ci sono due item del tipo:
 - $S \rightarrow a.$; $B \rightarrow a.$
 - In questo caso, nella tabella di parsing le celle presentano due reduce possibili.

Scegliere una delle due azioni non è deterministico, in quanto ciascuna, per come è costruito una tabella di parsing, è potenzialmente corretta. Quindi bisogna cambiare la grammatica o bisogna cambiare il parser bottom up, aumentando la sua complessità a SLR(k), LR(k+1) o LALR(k).

Si può andare ad aiutare la scelta di uno dei due algoritmi analizzando i prefissi viabili e capendo quale scelta ci porta ad un prefisso viabile.

101. Cos'è un prefisso viabile? Come lo si definisce in termini di una grammatica?

Sia la grammatica $G = (NT, T, R, S)$. Un prefisso viabile per la grammatica G è:

$$\gamma \in (T \cup NT)^* : S \Rightarrow^* \delta A \gamma \Rightarrow \delta \alpha \beta \gamma = \gamma \beta \gamma, \quad \text{per qualche } \gamma, \beta \in (T \cup NT)^* \text{ e } A \rightarrow \alpha \beta \in R$$

S è un prefisso viabile per definizione.

Se $\beta = \varepsilon$ allora γ si dice prefisso viabile completo e α è una maniglia per $\gamma\gamma$, quindi ci posso eseguire una reduce.

Essenzialmente è una stringa composta da terminali e nonterminali che si trova a destra di una possibile produzione, anche incompleta, che è prefisso di $\beta\gamma$.

Si dice che un prefisso viabile è una stringa che può apparire sulla pila di un parser bottom up per una computazione che accetta un input, ovvero "qualcosa di corretto".

102. Cos'è un item LR(0)? Come si generano tutti gli item di una grammatica (aumentata con un simbolo iniziale nuovo S')?

Un item in LR(0) è una produzione con un delimitatore, spesso un punto, che si può interporre tra qualsiasi lettera della parte destra della produzione.

Il punto ci va ad indicare quale parte della produzione è già stata letta, alla sua sinistra, e quale deve ancora essere letta per fare la reduce, a destra.

Si creano facendo un nuovo item per ogni posizione dove si può mettere un punto nelle produzioni della grammatica.

103. Com'è fatto il NFA dei prefissi viabili? Come si ricava il DFA dei prefissi viabili, detto anche automa canonico LR(0)?

L'NFA è fatto usando il seguente algoritmo:

$$q_0 = \{S' \rightarrow .S\}$$

$$Q = \{q_0\}$$

Fino a quando non ci sono più modifiche:

Sia X uno stato con item $Y \rightarrow a.Xb$:

Se $X \in NT, \forall X \rightarrow \gamma$ si aggiunge un nuovo stato con dentro l'item $X \rightarrow .\gamma$ e $\delta(X, \epsilon) = \text{nuovo stato}$

Se $X \in T \cup NT$ si aggiunge un arco $\delta(R, X) = \text{stato con } [Y \rightarrow aX.b]$

Il DFA dei prefissi viabili può essere ottenuto a partire dall'NFA usando due modi:

- La costruzione per sottoinsiemi vista quando abbiamo visto i linguaggi regolari
- Attraverso un metodo diretto, per il quale serve definire $Clos(X)$ e $Goto(X, y)$

Definiamo quindi $Clos(X)$:

Ripeti fino a quando ci sono cambiamenti in X :

Per ogni produzione $A \rightarrow a.Yb$

Per ogni produzione $Y \rightarrow \gamma$

aggiungi $Y \rightarrow .\gamma$ come item di X

Ritorna X

Definiamo ora il $Goto(X, y)$

$$J = \emptyset$$

Per ogni item $A \rightarrow a.yb$ in X :

aggiungi $A \rightarrow ay.b$ in J

Ritorna $Clos(J)$

Per la costruzione immediata diciamo:

Definiamo lo stato iniziale $I_0 = Clos(S' \rightarrow .S)$

Definiamo l'insieme degli stati $Z = \{I_0\}$

Ripeti fino a quando cambia Z o δ

Per ogni item $A \rightarrow a.Xb$ dello stato Y

$J = Goto(Y, X)$

$\delta(Y, X) = J$

Aggiungi J a Z

104. Come è fatta una tabella di parsing LR(0)? Come la si riempie a partire dall'automata canonico LR(0)? Quando una grammatica è di classe LR(0)?

Una tabella di parsing LR(0) è formata da:

- Le colonne sono indicizzate da $T \cup NT \cup \{\$ \}$
- Le righe sono indicizzate dagli stati del DFA
- Le celle vengono riempite nel seguente modo, usando il DFA:
 - o Con l'arco $s \xrightarrow{n} t$ con $n \in T$ si fa *shift* t in $M[s, n]$
 - o Con l'arco $s \xrightarrow{n} t$ con $n \in NT$ si fa *goto* t in $M[s, n]$
 - o Se in uno stato s c'è un una reduce, ovvero un item del tipo $A \rightarrow a$. Allora $\forall x \in T \cup \{\$ \}$ in $M[s, x]$ si scrive *reduce* s .
 - o Se $S' \rightarrow S$ in s allora *accept* in $M[s, \$]$

Una grammatica è di classe LR(0) quando si riesce a creare una tabella di parsing con al più una operazione per ogni cella.

105. Come è fatto il parser LR(0) che utilizza la tabella di parsing LR(0)? Quanti stack servono?

Un parser LR(0) può venire descritto, in un suo stato istantaneo, da una 3-pla:

$$(s_1 \dots s_n, x_1 \dots x_k, a_1 \dots a_j)$$

Dove $s_1 \dots s_n$ sono gli stati percorsi del DFA

$x_1 \dots x_k$ sono gli elementi attualmente sulla pila

$a_1 \dots a_j$ è l'input ancora da leggere.

Esso legge dalla pila in input il prossimo input, valutando cosa fare usando lo stato attuale sulla pila degli stati e confrontandosi quindi con il DFA.

Nel caso decidesse di fare uno shift, allora sposterebbe l'input da una pila all'altra e cambierebbe lo stato.

Nel caso decidesse di fare un reduce, rimuoverrebbe tanti oggetti dalla pila dei simboli attuali quanto la lunghezza della produzione e altrettanti stati dallo stack degli stati del DFA.

Nel caso facesse un accept, allora dovrebbe avere esattamente lo stato con l'accept sulla pila degli stati e $S\$$ sulla pila dei simboli e $\$$ sulla pila in input.

All'interno di questa descrizione didattica vengono usate due pile.

Nella pratica, viene usata solo la pila contenente gli stati percorsi dal DFA, in quanto non gli serve una "visualizzazione", come a noi che la eseguiamo a mano, dello stato degli elementi letti, in quanto questo viene pressoché memorizzato da come ci stiamo muovendo nel DFA.

106. Esistono grammatiche non LR(0)? Fare un esempio semplice.

Un esempio di grammatica non LR(0) è la grammatica:

$$\begin{aligned} S' &\rightarrow S \\ S &\rightarrow a \mid ab \end{aligned}$$

Vi è un conflitto shift-reduce, tra fare lo shift o ridurre quando si legge a .

107. Come è fatta e come si riempie una tabella di parsing SLR(1)? Cosa vuol dire l'acronimo SLR? Perché si mette 1 come parametro?

Si parte innanzitutto dal DFA canonico per LR(0). Si riempiono le cose come per LR(0), tutto tranne per le reduce, che vengono inserite nel seguente modo:

$$[A \rightarrow a.] \in I \Rightarrow M[I, x] \quad \forall x \in \text{Follow}(A) \text{ e } A \neq S$$

La S sta per *simple*, in quanto è un modo semplificato di aggiungere un lookahead. Si mette 1 come parametro perché si considera un lookahead per fare le reduce.

108. Quando una grammatica è di classe SLR(1)? Esistono grammatiche non di classe SLR(1)?

Una grammatica è di classe SLR(1) quando la tabella di parsing non presenta due scelte all'interno di un'unica cella. Esistono grammatiche che non sono di classe SLR(1).

109. Cos'è un item LR(1)? Come si costruire il NFA LR(1)? E come l'automa canonico LR(1)?

Un item LR(1) è composto da due parti:

- Il nucleo, che è definito esattamente come un item di LR(0), ovvero è una produzione con un delimitatore, spesso un punto, che separa la parte letta e la parte non letta della produzione e si può trovare in qualsiasi suo punto.
- Il simbolo di lookahead, il quale ci permette di stabilire cosa si dovrebbe trovare dopo la produzione.

Per costruire un NFA LR(1) si usa il seguente algoritmo:

Si definisce lo stato iniziale come $[S' \rightarrow .S, \$]$

Si ripete la seguente operazione fino a quando non ci sono più cambiamenti:

$$\begin{aligned} &\forall X \in T \cup NT \text{ tali che lo stato } S \text{ sia l'item } [A \rightarrow a.Xb, y] \\ &\delta(S, X) = J \text{ dove } J \text{ ha un item } [A \rightarrow aX.b, y] \\ &\forall X \in NT \text{ tali che lo stato } S \text{ sia l'item } [A \rightarrow a.Xb, y] \text{ e } \forall X \rightarrow \gamma \\ &\delta(S, \epsilon) = J \text{ dove } J \text{ ha un item } [X \rightarrow \gamma, \text{First}(b \cdot y)] \end{aligned}$$

Per costruire un DFA LR(1) si devono usare le due funzioni ausiliare $\text{Clos}(I)$ e $\text{Goto}(I, X)$

$\text{Clos}(I)$:

```

J = ∅
Esegui fino a quando cambiamenti in J
  Sia Z = [A → a.Xb, y] un item di I:
    Per ogni X → γ ∈ R
      Per ogni β ∈ First(by)
        Add [X → γ, β] a J
Ritorna J

```

$\text{Goto}(I, X)$:

```

J = ∅
∀Z item = [A → a.Xb, y] di I:
  Aggiungi Clos(A → aX.b, y) a J
Ritorna J

```

Algoritmo di costruzione

$$I_0 = \text{Clos}([S' \rightarrow \cdot S, \$])$$

$$\delta = \emptyset, \quad S = \{I_0\}$$

Finche δ e S vengono modificati:

Sia K un elemento di S

Sia $[A \rightarrow a.Yb, y]$ un item di K

Sia $L = \text{Goto}(Y, X)$

Aggiungi L a S

$$\delta(K, Y) = L$$

110. Come è fatta e come si riempie una tabella di parsing LR(1)?

Una tabella di parsing LR(1) ha:

- Le colonne indicizzate con $T \cup NT \cup \$$
- Le righe indicizzate con gli stati del DFA dei prefissi viabili (automa canonico)
- Le celle vengono riempite nel seguente modo:
 - Se $s \xrightarrow{x} t$ e $x \in NT$, allora $M[s, x] = \text{goto } t$
 - Se $s \xrightarrow{x} t$ e $x \in T$, allora $M[s, x] = \text{shift } t$
 - Se lo stato s ha l'item $[S' \rightarrow \cdot, \$]$, $M[s, \$] = \text{accept}$
 - Se lo stato s , e sia n la produzione $A \rightarrow a$, ha l'item $[A \rightarrow a \cdot, y]$, $M[s, y] = \text{reduce } n$

111. Come è fatta una tabella di parsing LALR(1)? Quando una grammatica è di classe LALR(1)?

Una tabella di parsing LALR(1) viene fatta nello stesso modo delle altre tabelle di parsing. LALR(1) deriva dal fondere insieme gli stati che hanno gli stessi nuclei. In questo modo, si ottengono un numero di stati uguali all'LR(0), ma si hanno molte meno cose non deterministiche.

Una grammatica è di classe LALR(1) quando non vi sono più di due produzioni per la stessa cella della tabella di parsing.

112. Esistono grammatiche LALR(1) che non sono SLR(1)? Esistono grammatiche LR(1) che non sono LALR(1)? Mostrare una grammatica che è LR(1) ma non LALR(1).

Sì, sì.

$$G = \begin{cases} S \rightarrow aAa \\ S \rightarrow bAb \\ S \rightarrow aBb \\ S \rightarrow bBa' \\ A \rightarrow c \\ B \rightarrow c \end{cases} \quad \text{questa qua esce LR(1) ma non LALR(1)}$$

113. Come si può generalizzare l'idea per ogni $k \geq 2$? Ovvero, quando una grammatica è di classe LR(k), SLR(k) o LALR(k)?

Per generare LR(k):

- Item $LR(k) = [item\ LR(0), \beta]$, con $|\beta| \leq k$
- Item iniziale = $[S' \rightarrow .S, \$]$
- Quando $[A \rightarrow a.X\gamma, \beta] \in S$ (Stato dell'automa canonico $LR(k)$), allora pure $[X \rightarrow .\delta, w] \in S$, se $X \rightarrow \delta$ è una produzione e $w \in First_k(\gamma\beta)$
- Colonne su T^k

Grammatiche SLR(k):

- Si parte dall'automa canonico $LR(0)$ e si riempie la tabella di parsing SLR(k) che ha colonne su T^k secondo la legge:
 - o $[A \rightarrow a.] \in S$ e $A \neq S'$, inserisco "reduce $A \rightarrow a$ " in $M[S, w]$, $\forall w \in Follow_k(A)$

Grammatiche LALR(k)

- Si parte dall'automa canonico $LR(k)$ e si fondono insieme gli stati con lo stesso nucleo.

114. Come si relazionano le grammatiche della famiglia LR (LR,SRL,LALR) al variare di k? Come si relazionano le grammatiche $LR(k)$ e $LL(k)$ al variare di k? Le grammatiche $LR(k)$ e $LL(k)$ sono sempre non ambigue? Esistono grammatiche ambigue che sono $LL(k)$ o $LR(k)$ per qualche k? Esistono grammatiche che non sono $LR(k)$ per nessun k?

$$SLR(k) \subset LALR(k) \subset LR(k)$$

$$LL(k) \subset LR(k), \quad LL(k) \text{ non sono } \subset LR(k)$$

LL e LR sono sempre non ambigue.

No, non sono mai ambigue.

Sì, quelle ambigue non sono mai $LR(k)$ per nessun k.

115. Come si relazionano i linguaggi della famiglia $LR(k)$ rispetto a quelli $LL(k)$? Esistono linguaggi liberi deterministici che non sono $LR(k)$ per qualche k? Esistono linguaggi liberi deterministici che non sono $LL(k)$ per qualche k?

Scritto sopra.

No, tutti i linguaggi liberi deterministici posso avere grammatiche $SLR(1)$.

Sì, esistono linguaggi liberi deterministici che non sono $LL(k)$ per nessun k.

116. Esiste un linguaggio regolare che non è $LR(0)$? Come si relazionano i linguaggi $LR(0)$ rispetto a quelli $LL(1)$? La prefix property è una condizione necessaria e sufficiente affinché un linguaggio sia di classe $LR(0)$?

Sì, per esempio il linguaggio $\{a, ac\}$ non è $LR(0)$ ma è un linguaggio regolare in quanto finito.

I linguaggi $LR(0)$ hanno elementi in comune e non in comune con quelli $LL(1)$, nessuno dei due è strettamente sottoinsieme dell'altro.

$L \text{ libdet} \wedge \text{PrefixP}(L) \Rightarrow L \text{ è LR}(0)$

$L \text{ è LR}(0) \text{ e finito} \Rightarrow L \text{ gode della prefix property}$

$L \text{ non gode della prefix property ed è finito} \Rightarrow L \text{ non è LR}(0)$

$L \text{ è LR}(0) \text{ e infinito} \Rightarrow L \text{ può non godere della prefix property.}$

117. La classe dei linguaggi SLR(1) coincide con la classe dei linguaggi liberi deterministici?

Sì.

118. Cos'è YACC? Quale è il suo input e qual è il suo output? Come si ottiene un parser eseguibile a partire da un file .y? Come agisce il parser generato da YACC in sintonia con lo scanner generato da Lex?

YACC è un generatore di analizzatori sintattici. Prende in input un file .y che contiene la descrizione della grammatica libera e dà in output un file C che realizza il parser LALR(1) con una costruzione diretta non vista a lezione.

Si ottiene un eseguibile dando i file .c che risultato da YACC ad un compilatore di c, indicando le varie librerie che deve usare.

L'analizzatore sintattico generato da YACC usa l'analizzatore lessicale come subroutine, chiedendoli per esempio il token successivo e gestendo alcune variabili in comune, come *yylval*, permettendo quindi scambiarsi informazioni tra di loro.

119. Come è la struttura di un file .y di YACC? Cosa sono le regole? Cos'è l'azione semantica?

Un file YACC è formato nel seguente modo:

%{Prologo%}

- Parte opzionale che contiene definizioni di macro e altre dichiarazioni di variabili o funzioni che saranno usate nelle sezioni seguenti.
- Viene copiato da YACC nell'output (*y.tab.c*) in modo da precedere la definizione della funzione "yyparse" che effettivamente farà l'analizzatore sintattico

Definizioni

- Contiene dichiarazioni di simboli usati nella descrizione della grammatica (nomi di token, condivisioni con lex)
- In questa sezione è possibile dichiarare la precedenza e l'associatività di alcuni termini / operatori

%

Regole

- Una produzione della forma
 - o $\text{nonterm} \rightarrow \text{corpo}_1 \mid \dots \mid \text{corpo}_k$
- È espressa in YACC con le regole:
 - o $\text{nonterm} : \text{corpo}_1 \{ \text{azione semantica}_1 \}$
| ...

| $corpo_k \{azione\ semantic_k\}$
;

- Il nonterminale è una stringa alfanumerica non dichiarata in precedenza come token
 - o Un carattere tra singoli apici, 'a', è un terminale
 - o Il simbolo iniziale della grammatica è il nonterminale usato nella prima regola
- Le azioni semantiche sono codice C eseguito al momento in cui il parser riduce mediante quella produzione.
 - o Calcola il "valore semantico" della testa della "produzione", in funzione dei valori semantici dei simboli che compongono il corpo. Il "valore semantico" può essere:
 - L'albero di derivazione, nel caso in cui stiamo producendo un compilatore che genera alberi esplicitamente
 - Il codice intermedio, connesso alla produzione, se andiamo direttamente a produrre codice intermedio
 - La vera e propria valutazione dell'espressione, se stiamo producendo un interprete.
 - o Si può usare \$i per usare l'i-esimo valore trovato nella produzione.

%

Funzioni ausiliarie

- Contiene le funzioni di supporto per la generazione del parser, tra questi yylex(), funzione che invoca l'analizzatore lessicale e che restituisce il nome del token (che deve essere stato definito nella sezione "definizioni" di YACC) e il suo valore (nella variabile yylval)
- Deve o contenere yylex() con la sua definizione
- Oppure, se viene generato con Lex, allora deve fare #include "lex.yy.c"

120. È possibile gestire grammatiche ambigue con YACC, specificando le associatività e le priorità fra gli operatori per risolvere l'ambiguità? Come si comporta YACC in presenza di conflitti?

Sì, queste si possono specificare all'interno delle definizioni, ovvero la parte che sussegue il prologo e precede le regole. Possiamo indicare con la parola chiave *left* o *right* l'associatività rispettivamente destra o sinistra dei nonterminali che desideriamo.

In generale, in caso manchino le indicazioni su come risolvere conflitti:

- Risolve shift/reduce a favore di shift
- Risolve reduce/reduce a favore della produzione elencata prima

Domande 3

mercoledì 14 giugno 2023 23:07

- 121. È possibile costruire un programma Check che, preso in input un qualunque programma P, restituisce 1 se P è corretto e 0 se P è scorretto? Ovvero esiste un qualche compilatore che può scovare tutti i possibili errori di un programma?**

Non si può fare.

Assumiamo per assurdo che si possa costruire un programma che controlla 1 dei parametri della correttezza di un programma, ovvero il suo terminare. Questo programma ritorna 1 se termina per ogni output e ritorna 0 altrimenti:

$$H(P, x) = \begin{cases} 1 & \text{se } P(x) \downarrow \\ 0 & \text{se } P(x) \uparrow \end{cases}$$

Supponiamo ora di creare un secondo programma che mi ritorna 0 se converge e per una x e 1 se diverge per una x:

$$K(P) = \begin{cases} 1 & \text{se } P(P) \downarrow \\ 0 & \text{se } P(P) \uparrow \end{cases}$$

Ora, diamo in pasto a F H e P:

$$G(P) = \begin{cases} 1 & \text{se } K(P) = 0 \\ \uparrow & \text{se } K(P) = 1 \end{cases}$$

Ma se noi a G diamo in input G, allora essa ritorna uno sse essa diverge e diverge sse essa converge. Quindi è assurdo. Quindi nulla di questo può esistere.

- 122. Cosa dice il problema della fermata (Halting problem)? (L'errore in esame è la possibilità di non terminare il calcolo). Come si dimostra che il problema non può essere risolto?**

Il Halting problem è un problema relativo al capire se è possibile capire a priori quando un programma terminerà e quando invece diverge.

Supponiamo di riuscire a creare un programma che ritorna 1 se il programma converge e ritorna 0 se il programma diverge, che è un programma che fa la funzione che noi vogliamo.

$$H(P, x) = \begin{cases} 1 & \text{se } P(x) \downarrow \\ 0 & \text{se } P(x) \uparrow \end{cases}$$

Ora, con la conoscenza di scrivere questo programma, noi possiamo andare a creare un altro programma ausiliare, ovvero:

$$F(P) = \begin{cases} 1 & \text{se } P(P) \downarrow \\ 0 & \text{se } P(P) \uparrow \end{cases}$$

Infine, possiamo creare un ultimo programma, visto che siamo riusciti a fare questi due, del tipo:

$$G(P) = \begin{cases} 1 & \text{se } F(P) = 0 \\ \uparrow & \text{se } F(P) = 1 \end{cases} \Rightarrow G(G) \uparrow \Leftrightarrow F(P) = 1 \Leftrightarrow G(G) \downarrow$$

123. Quando un problema è decidibile?

Un problema è decidibile se si può rispondere sia di vero, che di falso, in tempo finito. Ovvero la soluzione vera o falsa converge in tempo finito.

Un problema semi decidibile se si può rispondere vero in tempo finito, ma non si può rispondere a falso in tempo finito.

124. Quali sono tipici esempi di proprietà indecidibili per i linguaggi di programmazione?

Esempi di proprietà indecidibili sono:

Terminazione, ovvero ritornare vero se converge e falso se diverge

Divergenza, ovvero vero se diverge e falso se converge

Equivalenza

Calcolo di una costante, ovvero se P calcola un valore costante per ogni input

Generazione di errori *run-time*

125. Cos'è una macchina di Turing (MdT)? Che cosa calcola?

Una macchina di Turing è una 6-upla formata da:

$$(Q, A, B, q_0, q_f, \delta)$$

Q sono stati finiti

A è l'alfabeto finito in input

B è l'alfabeto finito del nastro, super insieme di quello in input, ed ha la casella vuota

q_0 è lo stato iniziale

q_f è lo stato finale

$\delta: Q \times B \rightarrow Q \times B \times \{sx, dx\}$

Una macchina di Turing può essere vista come una macchina che calcola funzioni parziali binarie, ovvero data una stringa, ritorna 1 se si arriva in uno stato finale, 0 se non si arriva in uno stato finale ma non si può più continuare, oppure diverge.

126. Quando un linguaggio è detto Turing-completo? Cosa afferma la tesi di Church-Turing e perché non si può dimostrare?

Un linguaggio è detto Turing completo se può calcolare tutte le funzioni che calcolabili da una macchina di Turing.

La tesi di Church-Turing afferma che:

- Se una funzione può essere calcolata algebricamente in un qualche formalismo, allora è calcolabile con MdT.
- Non è possibile verificarla in quanto non c'è una definizione formale di cosa voglia dire "algebricamente", pertanto bisognerebbe farlo per ogni formalismo, ma ci sarebbero infinite prove e chiunque potrebbe proporre nuovi formalismi da dimostrare.

127. I normali linguaggi di programmazione sono Turing completi? Cosa afferma il teorema di Jacopini-Bohm?

I normali linguaggi sono Turing completi se si assume che la memoria che usano è sufficiente per eseguire il programma / infinita in generale.

Se un linguaggio prevede almeno le 4 operazioni:

- Iterazione indeterminata o ricorsione
- *If – then – else*
- Concatenazione dei comandi
- Istruzioni di assegnamento

Allora suddetto linguaggio è Turing completo.

128. Quale relazione esiste tra espressività di un formalismo e decidibilità di proprietà dello stesso? Fare una panoramica prendendo in esame i 3 formalismi MdT, PDA, DFA e le due proprietà $w \in L(m)$ (Ovvero è w riconosciuta dalla macchina M ?) e $L(M_1) = L(M_2)$ (Le due macchine sono equivalenti, ovvero riconoscono lo stesso linguaggio?)

Più si vanno a diminuire le possibilità espressive di una grammatica e più proprietà vanno ad assumere da questo punto di vista.

Per le macchine di Turing, entrambe le proprietà sono indecidibili.

Per i PDA è decidibile l'appartenenza ma non l'uguaglianza.

Per i DFA è riconoscibile sia l'appartenenza che l'uguaglianza.